

RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM - 602 105



**MA23435 PROBABILITY, STATISTICS
AND SIMULATION**

Laboratory Record Note Book

Name :

Year / Branch / Section :

University Register No. :

College Roll No.:

Semester :

Academic Year :



RAJALAKSHMI ENGINEERING
COLLEGE

An Autonomous Institution

BONAFIDE CERTIFICATE

Name:

Academic Year: Semester: Branch:

Register No :

*Certified that this is the bonafide record of work done by the above student in
the Laboratory
during the academic year 202 - 202*

Signature of Faculty in-charge

Submitted for the Practical Examination held on.....

Internal Examiner

External Examiner

INDEX

[illegible]

Ex. No. 1	BASIC FUNCTIONS IN R AND PLOTTING
Date : 24.01.2026	

1.Problem Statement:

Use the sqrt() function to calculate the square root of 99.

Aim:

To compute the square root of a given number using the built-in sqrt() function in R.

Procedure:

1. Enter the number 99 in the sqrt() function
2. Execute the command to perform the calculation.
3. Note down the output displayed (square root of 99).

Program:

```
result<-sqrt(99)
print(result)
```

Output:

9.949874

2.Problem Statement:

Create a boxplot for random normal data.

Aim:

To create a boxplot for randomly generated normal data using R.

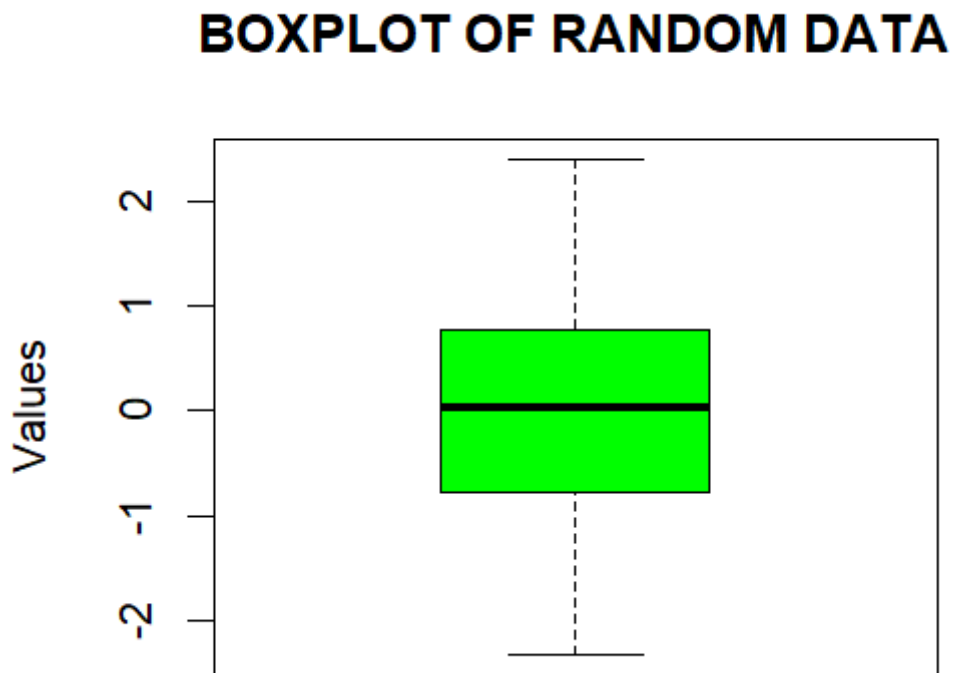
Procedure:

1. Generate random normal data using rnorm() function.
2. Use the boxplot() function to create the boxplot.
3. Execute the command and observe the graphical output.

Program:

```
data<-rnorm(100)
boxplot(data,main="BOXPLOT OF RANDOM DATA",ylab="Values",col="green")
```

Output:



3.Problem Statement:

Use the log10() function to compute the logarithm of 5000 to base 10.

Aim:

To compute the base-10 logarithm of a given number using the log10() function in R.

Procedure:

1. Enter the number 5000 in the log10() function as log10(5000).
2. Execute the command to perform the calculation.
3. Note down the output displayed (logarithm value).

Program:

```
result<-log10(5000)
print(result)
```

Output:

3.69897

4.Problem Statement:

Use the abs() function to compute the absolute value of -12.

Aim:

To compute the absolute value of a given number using the abs() function in R.

Procedure:

1. Enter the number -12 in the abs() function as abs(-12).
2. Execute the command to perform the calculation.
3. Note down the output displayed (absolute value).

Program:

```
result<- abs(-12)
print(res)
```

Output:

12

5.Problem Statement:

Create a pie chart of academic data of a class.

Aim:

To create a pie chart in R that visualizes the academic performance of a class by representing the distribution of different grade categories.

Procedure:

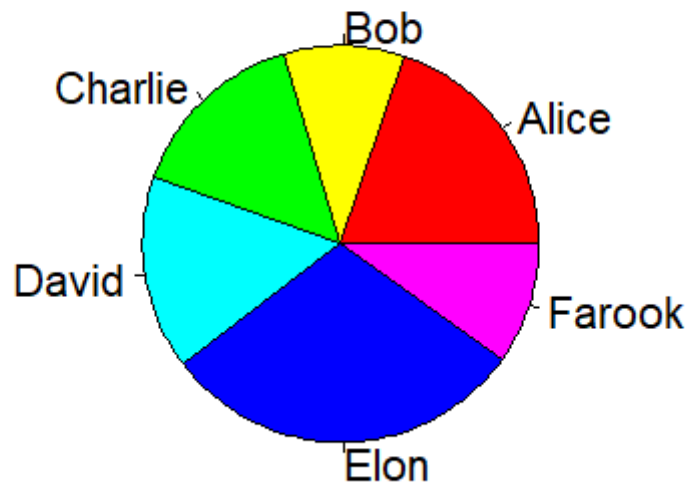
1. Define the academic data by creating vectors for grade labels and student counts.
2. Use the pie() function with the defined data, labels, and color arguments to generate the chart.
3. Execute the command to display the pie chart and observe the distribution of the academic data.

Program:

```
marks<-c(20,10,15,16,30,10)
labels<-c("Alice","Bob","Charlie","David","Elon","Farook")
pie(marks,labels=labels,col=rainbow(6),main="ACADEMIC DATA OF CLASS")
```

Output:

ACADEMIC DATA OF CLASS



6.Problem Statement:

Use the ceiling() function to compute the smallest integer greater than or equal to 7.5.

Aim:

To compute the smallest integer greater than or equal to 7.5 using the ceiling() function in R.

Procedure:

1. Enter the number 7.5 in the ceiling() function as ceiling(7.5).
2. Execute the command to perform the calculation.
3. Note down the output displayed (smallest integer greater than or equal to the value).

Program:

```
result<-ceiling(7.5)
print(result)
```

Output:

7.Problem Statement:

Use the tanh() function to compute the hyperbolic tangent of 2.5

Aim:

To compute the hyperbolic tangent of 2.5 using the tanh() function in R.

Procedure:

1. Enter the value 2.5 in the tanh() function as tanh(2.5).
2. Execute the command to perform the calculation.
3. Note down the output displayed (hyperbolic tangent value).

Program:

```
result<-tanh(2.5)
print(result)
```

Output:

0.9866143

8.Problem Statement:

Create a scatter plot for random data.

Aim:

To create a scatter plot in R using random data to visualize the relationship between two variables.

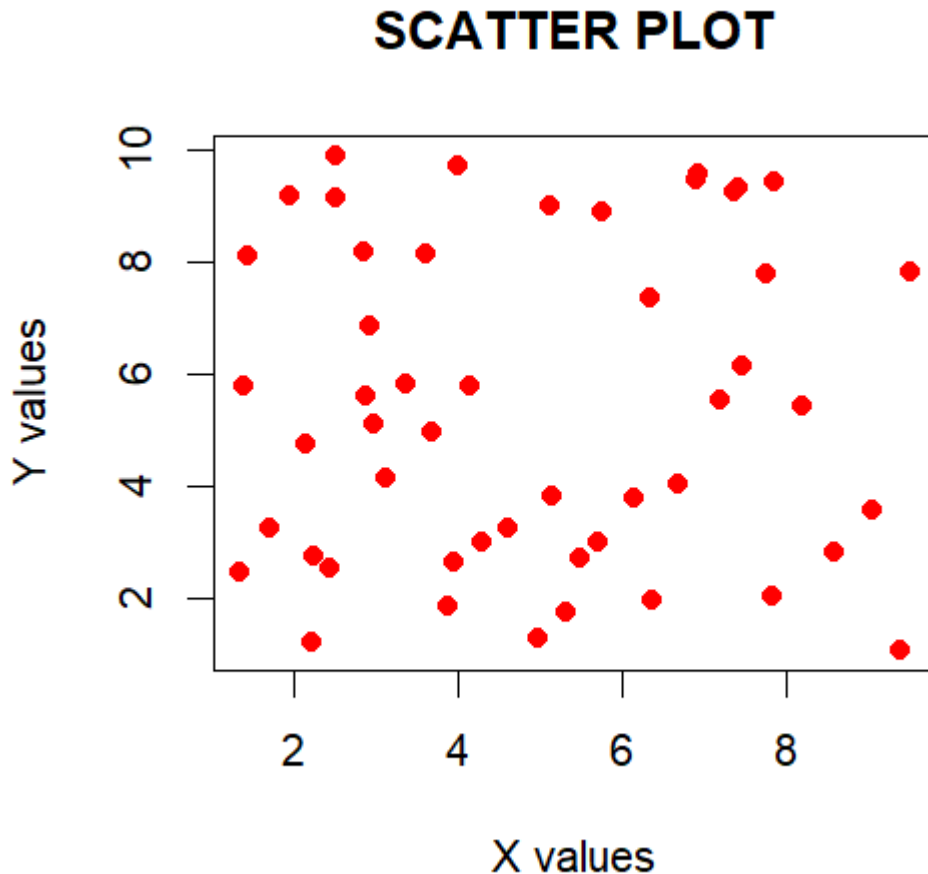
Procedure:

1. Generate two sets of random data for the x and y axes using the rnorm() or runif() function.
2. Use the plot() function and pass the random data sets as arguments to create the scatter plot.
3. Execute the command and observe the graphical output to see the distribution of the random points.

Program:

```
x<-runif(50,1,10)
y<-runif(50,1,10)
plot(x,y,main="SCATTER PLOT",xlab="X values",ylab="Y values",col="red",pch=16)
```

Output:



9.Problem Statement:

Define a custom function $f(x) = \cos(x)$ and integrate it from 0 to $\pi/2$ using the `integrate()` function.

Aim:

To define a custom function $f(x) = \cos(x)$ and calculate its definite integral from 0 to $\pi/2$ using the `integrate()` function in R.

Procedure:

1. Define the custom function `f <- function(x) cos(x)` to represent the mathematical expression.
2. Call the `integrate()` function by specifying the function `f`, the lower limit as 0, and the upper limit as $\pi/2$.
3. Execute the command to perform the integration and note down the resulting value displayed in the output.

Program:

```
f<-function(x) cos(x)
result<-integrate(f,lower=0,upper=pi/2)
print(result)
```

Output:

with absolute error < 1.1e-14

10.Problem Statement:

Create a line plot for the function $y = x^2 + 2x + 9$ over the range -5 to 5

Aim:

To create a line plot for the function $y = x^2 + 2x + 9$ over the range -5 to 5 in R.

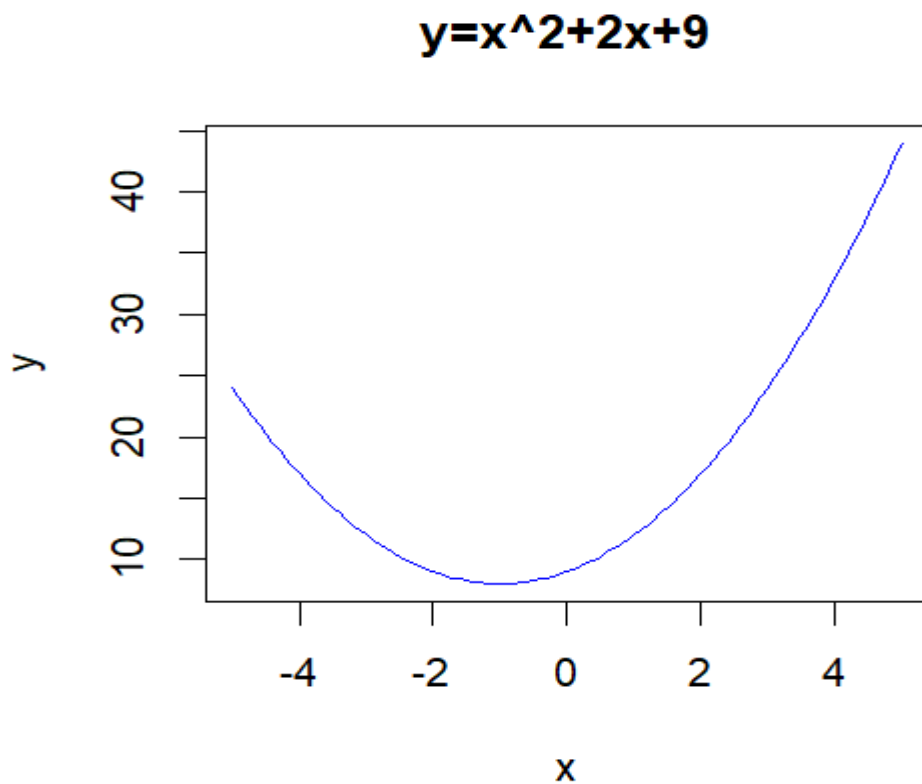
Procedure:

1. Create a sequence of values from -5 to 5 using the seq() function and assign it to the variable x.
2. Define the formula $y = x^2 + 2x + 9$ to calculate the corresponding y values for the defined range.
3. Use the plot() function with the argument type = "l" to generate and observe the line plot on the graph.

Program:

```
x<-seq(-5,5,by=0.1)
y<-x^2+2*x+9
plot(x,y,type="l",col="blue",main="y=x^2+2x+9",xlab="x",ylab="y")
```

Output:



Signature of the faculty

Ex. No. 2	MATHEMATICAL FUNCTIONS IN R - INTEGRATION
Date : 31.01.2026	

1.Problem Statement:

Use the cos() function to calculate the cosine of $\pi/8$.

Aim:

To compute the cos() function to calculate the cosine of $\pi/8$.

Procedure:

- 1.Use the cos() function to compute the cosine.
2. Pass pi /8 as an argument to the function.
3. Print the result using print().

Program:

```
result <- cos(pi / 8)
print(result)
```

Output:

0.9238795

2.Problem Statement:

Use the factorial() function to calculate the factorial of 90.

Aim:

To calculate the factorial of the given number using factorial() function in R.

Procedure:

1. Use the factorial() function to compute the factorial.
2. Pass the value 90 as an argument to the function.
3. Print the result using print().

Program:

```
r<-factorial(90)
r
```

Output:

1.485716e+138

3.Problem Statement:

Use the gcd() function from the gmp package to compute the greatest common divisor of 66 and 98.

Aim:

To compute the greatest common divisor of given number using the gcd() function in R.

Procedure:

1. Load the gmp library using library(gmp).
2. Use the gcd() function to compute the GCD of 66 and 98.
3. Print the result using print().

Program:

```
library(gmp)
r<-gcd(66,98)
r
```

Output:

2

4.Problem Statement:

Use the pracma package to evaluate the double integral of $f(x, y) = x^2 + y^3$ over the range 0 to 6 for both x and y.

Aim:

To evaluate the double integral of given function over the specified range for both x and y using pracma package in R.

Procedure:

1. Load the pracma library using library(pracma).
2. Define the function $f(x, y) = x^2 + y^3$
3. Use the dblquad() function to compute the double integral over the specified range.
4. Print the result using print().

Program:

```
library(pracma)
f<-function(x,y) x^2+y^3
r<-dblquad(f,0,6,0,6)
r
```

Output:

2376

5.Problem Statement:

Use the trunc() function to compute the truncated value of 9.8.

Aim:

To compute the truncated value of the given number using trunc() function in R.

Procedure:

1. Use the trunc() function to truncate the decimal part.
2. Pass the value 9.8 as an argument to the function.
3. Print the result using print().

Program:

```
r<-trunc(9.8)
r
```

Output:

9

6.Problem Statement:

Use the gamma() function to compute the gamma function for 15.

Aim:

To compute the gamma function for the given number using the gamma() function in R.

Procedure:

1. Use the gamma() function to compute the gamma value.
2. Pass the value 15 as an argument to the function.
3. Print the result using print().

Program:

```
r<-gamma(15)
r
```

Output:

87178291200

Signature of the faculty

Ex. No. 3	CONTROL FLOW LOOPS IN R
Date : 07.02.2026	

1.Problem Statement:

Square of 1st 20 natural numbers in a list using lapply.

Aim:

To compute the square of 1st 20 natural numbers in a list using lapply.

Procedure:

1. Create a list of numbers.
2. Use lapply() to apply the squaring function.

Program:

```
l<-1:20
s<-lapply(l,function(x) x^2)
s
```

Output:

```
[[1]] 1
[[2]] 4
[[3]] 9
[[4]] 16
[[5]] 25
[[6]] 36
[[7]] 49
[[8]] 64
[[9]] 81
[[10]] 100
[[11]] 121
[[12]] 144
[[13]] 169
[[14]] 196
[[15]] 225
[[16]] 256
[[17]] 289
[[18]] 324
[[19]] 361
[[20]] 400
```

2.Problem Statement:

Calculate the running average of a 6dimensional vector using a for loop.

Aim:

To calculate the running average of a 6dimensional vector using a for loop in R.

Procedure:

1. Create a vector of numbers.
2. Use a for loop to compute the cumulative sum and divide by the index.

Program:

```
v<-c(10,20,30,40,50,60)
ra<-c()
s<-0
for(i in 1:length(v)){
  s<-s+v[i]
  ra<-c(ra,s/i)
}
ra
```

Output:

10 15 20 25 30 35

3.Problem Statement:

Extract even numbers from a vector which is a list of 1000 to 1150.

Aim:

To extract even numbers from a vector in the specified list using R.

Procedure:

1. Create a vector of numbers from 1000 to 1150.
2. Use a for loop and if condition to identify even numbers.

Program:

```
n<-1000:1150
even<-c()
for(num in n){
  if(num%%2==0) even<-c(even,num)
}
even
```

Output:

1000 1002 1004 1006 1008 1010 1012 1014 1016 1018 1020 1022 1024 1026 1028 1030 1032 1034
1036 1038 1040 1042 1044 1046 1048 1050 1052 1054 1056 1058 1060 1062 1064 1066 1068 1070
1072 1074 1076 1078 1080 1082 1084 1086 1088 1090 1092 1094 1096 1098 1100 1102 1104 1106
1108 1110 1112 1114 1116 1118 1120 1122 1124 1126 1128 1130 1132 1134 1136 1138 1140 1142
1144 1146 1148 1150

4.Problem Statement:

Compute a cumulative sum until it exceeds 80.

Aim:

To compute the cumulative sum until it exceeds 80 using R.

Procedure:

1. Initialize variables for the sum and counter.
2. Use a while loop to add numbers until the sum exceeds 80.

Program:

```
s<-0
i<-1
while(s<=80){
  s<-s+i
  i<-i+1
}
s
```

Output:

91

5.Problem Statement:

Identify prime numbers between 100 and 2000 using a for loop.

Aim:

To compute the prime numbers between the specified range using for loop in R.

Procedure:

1. Use a loop to check divisibility for each number.
2. Print numbers that are prime.

Program:

```
is_prime<-function(n){
  if(n<2) return(FALSE)
  for(i in 2:sqrt(n)){
    if(n%%i==0) return(FALSE)
  }
  return(TRUE)
}
for(i in 100:2000){
  if(is_prime(i)) print(i)
}
```

Output:

```
101
103
107
109
113
127
.
.
.
.
.
.
1979
1987
1993
1997
1999
```

6.Problem Statement:

Print a multiplication table for numbers 10 to 20 using nested for loops.

Aim:

To print the multiplication table for numbers 10 to 20 using nested for loops in R.

Procedure:

1. Use two nested for loops to generate the table.
2. Print each product.

Program:

```
for(i in 10:20){
  for(j in 1:10){
    cat(i,"x",j,"=",i*j,"\n")
  }
}
```

Output:

```
10 x 1 = 10
10 x 2 = 20
10 x 3 = 30
.
.
.
.
.
.
.
20 x 8 = 160
20 x 9 = 180
20 x 10 = 200
```

7.Problem Statement:

Simulate rolling a die 25 times and print the results.

Aim:

To simulate rolling a die 25 times and print the results in R.

Procedure:

1. Use a for loop to simulate dice rolls using sample().
2. Print each roll.

Program:

```
for(i in 1:25){  
  r<-sample(1:6,1)  
  print(r)  
}
```

Output:

```
3  
5  
3  
3  
1  
4  
1  
1  
5  
3  
2  
2  
1  
6  
3  
4  
6  
1  
3  
5  
4  
2  
5  
1  
1
```

8.Problem Statement:

Compute a cumulative sum until it exceeds 90.

Aim:

To compute the cumulative sum until it exceeds 90 using R.

Procedure:

1. Initialize variables for the sum and counter.
2. Use a while loop to add numbers until the sum exceeds 90.

Program:

```
s<-0
i<-1
while(s<=90){
  s<-s+i
  i<-i+1
}
s
```

Output:

91

9.Problem Statement:

Perform matrix multiplication using nested for loops for the matrix

```
[1 2 3   [ 10 12 13
4 5 6     40 15 16
7 8 9] and 70 18 91]
```

Aim:

To perform matrix multiplication using nested for loops for the given matrices.

Procedure:

1. Define two matrices.
2. Use nested for loops to compute the product.

Program:

```
a<-matrix(c(1,2,3,4,5,6,7,8,9),nrow=3,byrow=TRUE)
b<-matrix(c(10,12,13,40,15,16,70,18,91),nrow=3,byrow=TRUE)
c<-matrix(0,nrow=3,ncol=3)
for(i in 1:3){
  for(j in 1:3){
    for(k in 1:3){
      c[i,j]<-c[i,j]+(a[i,k]*b[k,j])
    }
  }
}
c
```

Output:

```
      [,1] [,2] [,3]
[1,] 300  96 318
[2,] 660 231 678
[3,] 1020 366 1038
```

10.Problem Statement:

Generate the first 25 Fibonacci numbers using a for loop.

Aim:

To generate the first 25 fibonacci numbers using a for loop.

Procedure:

1. Initialize the first two Fibonacci numbers.
2. Use a for loop to compute the next numbers in the series.

Program:

```
n<-25
fib<-numeric(n)
fib[1]<-0
fib[2]<-1
for(i in 3:n){
  fib[i]<-fib[i-1]+fib[i-2]
}
fib
```

Output:

```
0  1  1  2  3  5  8  13  21  34  55  89  144  233  377  610  987  1597  2584
4181 6765 10946 17711 28657 6368
```

Signature of the faculty

Ex. No. 4	PROBABILITY DISTRIBUTIONS USING R- PDF, CDF FOR BINOMIAL, POISSON, EXPONENTIAL, UNIFORM AND NORMAL DISTRIBUTIONS
Date : 14.02.2026	

1.Problem Statement:

Plot the PDF and compute CDF of a Binomial distribution with parameters $n = 19$ and $p = 0.1235$.

Aim:

To compute the Cumulative Distribution Function (CDF) and plot the Probability Mass Function (PMF/PDF) for a Binomial distribution with $n = 19$ and $p = 0.1235$.

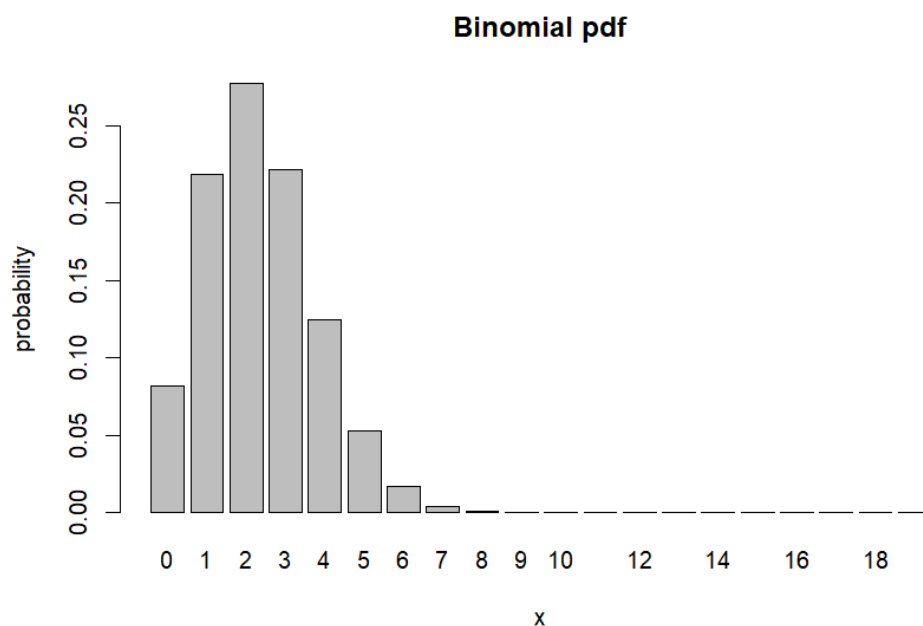
Procedure:

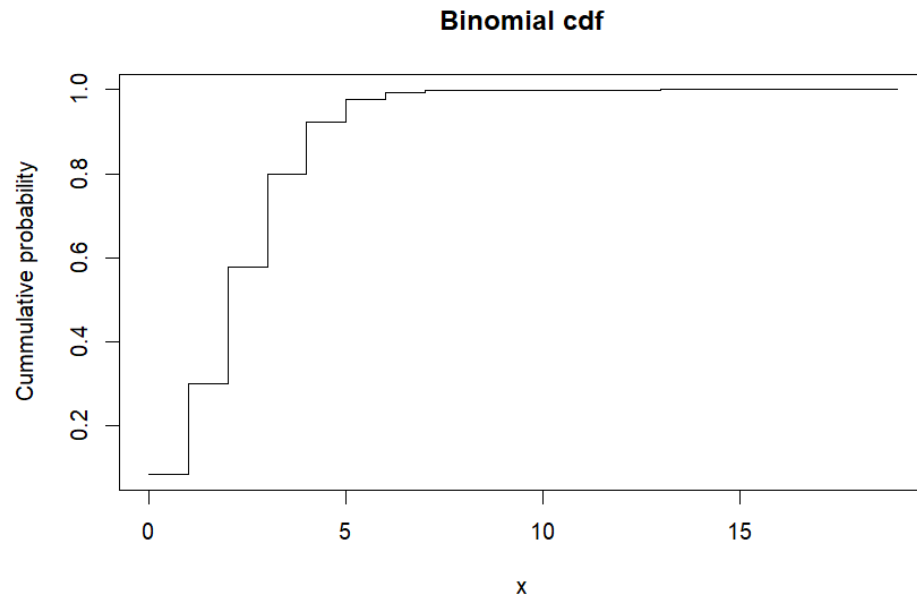
1. Define a sequence of possible successes (x from 0 to 19).
2. Use the `dbinom()` function to calculate the PMF/PDF and `pbinom()` for the CDF.
3. Use the `plot()` function to visualize the probabilities.

Program:

```
n<-19
p<-0.1235
x<-0:n
pdf<-dbinom(x,size=n,prob=p)
barplot(pdf,names.arg=x,main="Binomial pdf",xlab="x",ylab="probability")
cdf<-pbinom(x,size=n,prob=p)
plot(x,cdf,type="s",main="Binomial cdf",xlab="x",ylab="Cumulative probability")
```

Output:





2.Problem Statement:

Simulate 12 values from a Binomial distribution and compute the empirical mean and variance..

Aim:

To generate 12 random values from a Binomial distribution (assuming $n=10$, $p=0.5$ as parameters were not specified) and calculate their empirical mean and variance.

Procedure:

1. Use `rbinom()` to generate the 12 random numbers.
2. Use `mean()` and `var()` to calculate the empirical statistics of the simulated data.

Program:

```
set.seed(123)
n<-10
p<-0.5
data<-rbinom(12,size=n,prob=p)
mean(data)
var(data)
```

Output:

```
> mean(data)
[1] 5.5
> var(data)
[1] 2.636364
```

3.Problem Statement:

Calculate the probability of getting exactly 5 successes in a Binomial distribution with $n = 3$ and $p = 0.9$

Aim:

To calculate $P(X = 5)$ for a Binomial distribution where $n = 3$ and $p = 0.9$.

Procedure:

1. Use the `dbinom()` function.
2. The number of maximum possible successes ($n=3$), target successes ($x=5$).

Program:

```
n<-3
p<-0.9
prob<-dbinom(5,size=n,prob=p)
prob
```

Output:

0

4.Problem Statement:

Compute and plot the PDF & CDF for a Poisson distribution with $\lambda = 2$.

Aim:

To compute and plot the PMF/PDF and CDF for a Poisson distribution with rate parameter $\lambda = 2$

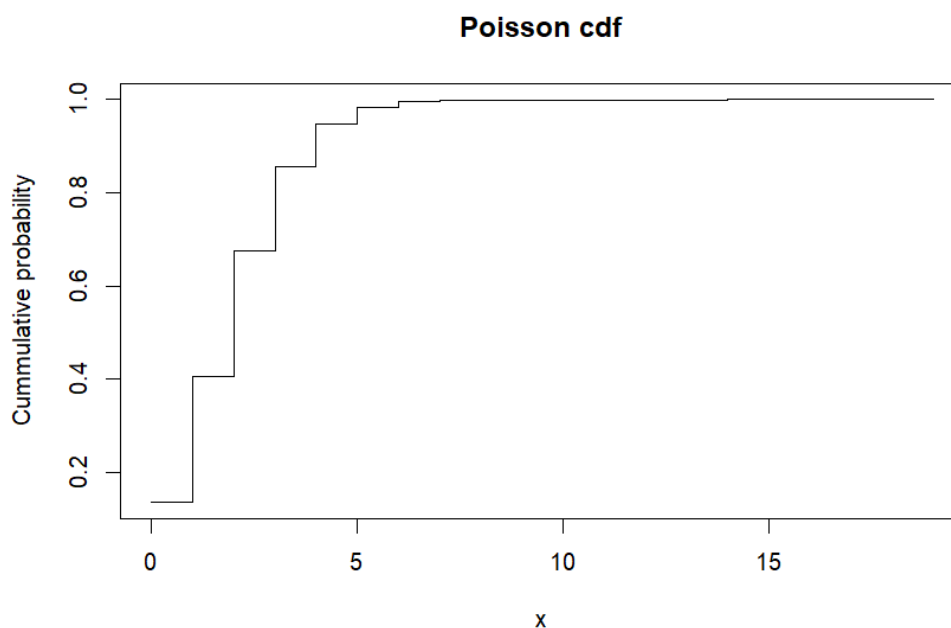
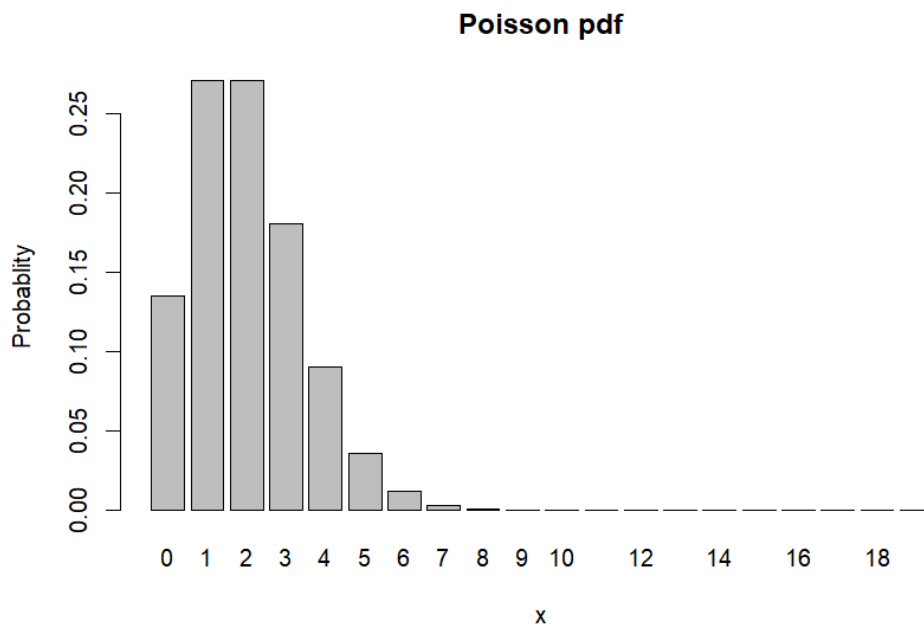
Procedure:

1. Define a sequence of values representing occurrences (x from 0 to 10).
2. Compute PDF using `dpois()` and CDF using `ppois()`.
3. Use `plot()` to map the distributions.

Program:

```
lambda<-2
X<-1:15
pdf<-dpois(x,lambda)
barplot(pdf,names.arg=x,main="Poisson pdf",xlab="x",ylab="Probablity")
cdf<-ppois(x,lambda)
plot(x,cdf,type="s",main="Poisson cdf",xlab="x",ylab="Cumulative probability")
```

Output:



5.Problem Statement:

Verify the mean and variance of a Poisson distribution with $\lambda = 1.5$ using simulation.

Aim:

To verify mathematically that the mean and variance of a Poisson distribution are both equal to λ (1.5) via large-scale simulation.

Procedure:

1. Generate a large sample (e.g., 100,000) using `rpois()`.
2. Compute the sample mean and variance and compare them against the theoretical value of 1.5.

Program:

```
set.seed(123)
data <- rpois(1000,lambda=1.5)
mean(data)
var(data)
```

Output:

```
> mean(data)
[1] 1.489
> var(data)
[1] 1.465344
```

6.Problem Statement:

Plot the PDF and CDF for a Uniform distribution on [0.5, 1.9]

Aim:

To compute and plot the Continuous Uniform PDF and CDF within the limits [0.5, 1.9].

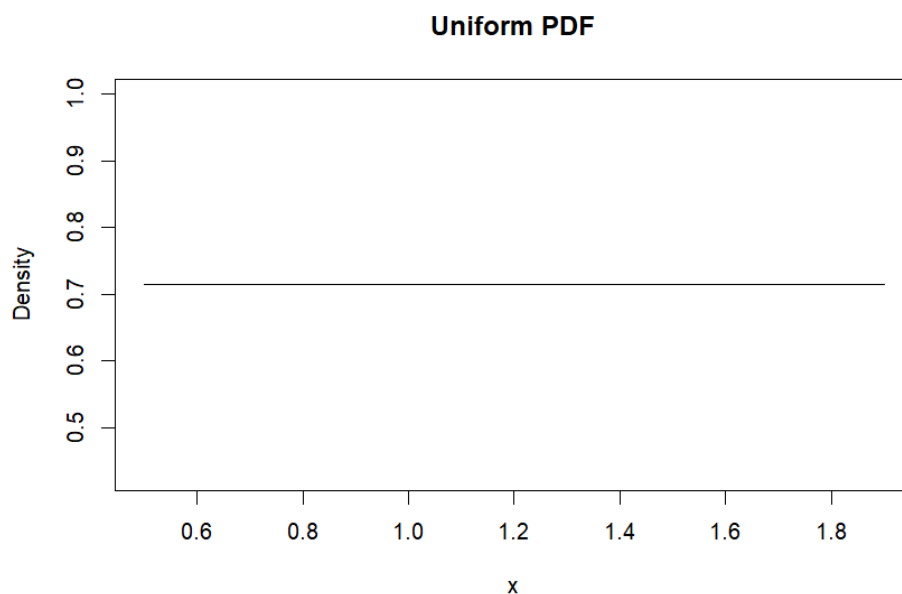
Procedure:

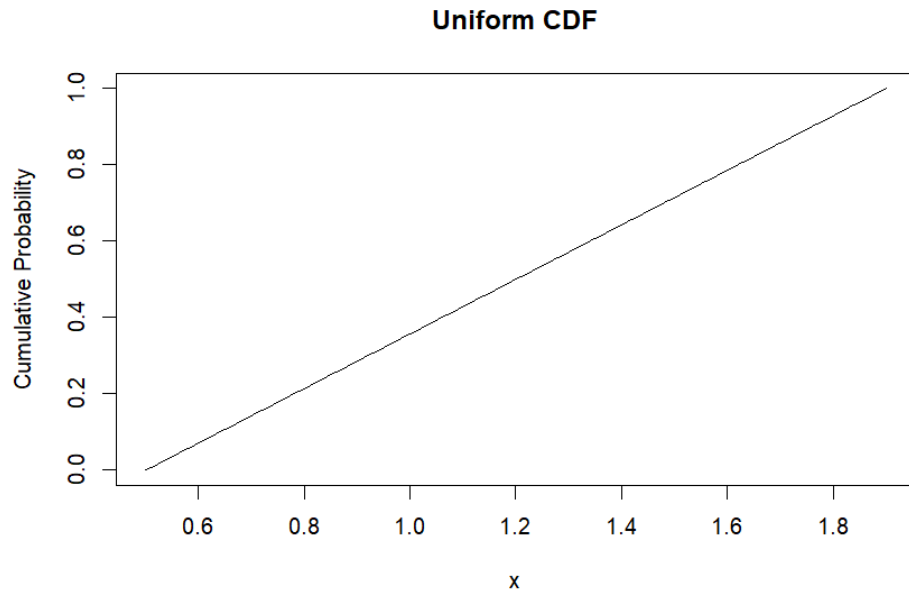
1. Create a fine continuous sequence of x values from 0 to 2.5 to show the boundaries.
2. Use dunif() for the PDF and punif() for the CDF.

Program:

```
a <- 0.5
b <- 1.9
x <- seq(a, b, by = 0.01)
pdf <- dunif(x, min = a, max = b)
plot(x, pdf, type = "l", main = "Uniform PDF", xlab = "x", ylab = "Density")
.cdf <- punif(x, min = a, max = b)
plot(x, .cdf, type = "l", main = "Uniform CDF", xlab = "x", ylab = "Cumulative Probability")
```

Output:





7.Problem Statement:

Generate 90 random numbers from a Uniform distribution between 150 and 180 and plot a histogram.

Aim:

To randomly simulate 90 values from Uniform(150, 180) and visualize their frequency distribution.

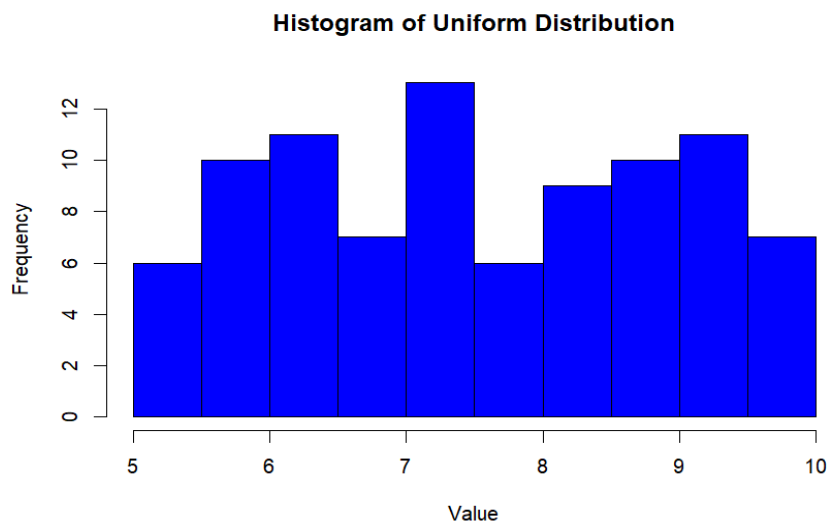
Procedure:

1. Generate the data using `runif(n, min, max)`.
2. Plot it using `hist()`.

Program:

```
set.seed(123)
data <- runif(90, min = 5, max = 10)
hist(data, main = "Histogram of Uniform Distribution", xlab = "Value", ylab = "Frequency", col = "blue")
```

Output:



8.Problem Statement:

Plot the PDF and CDF for a Normal distribution with mean = 3 and sd = 6.

Aim:

To calculate and plot the PDF and CDF for $N(\text{mean}=3, \text{sd}=6)$.

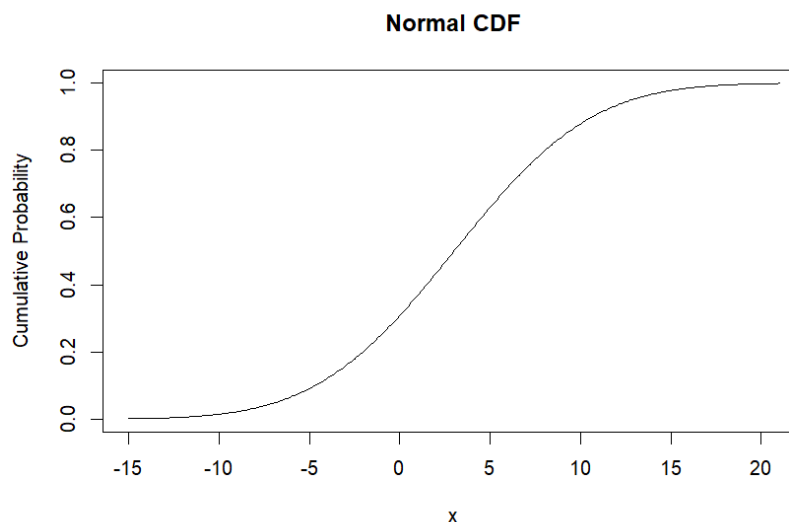
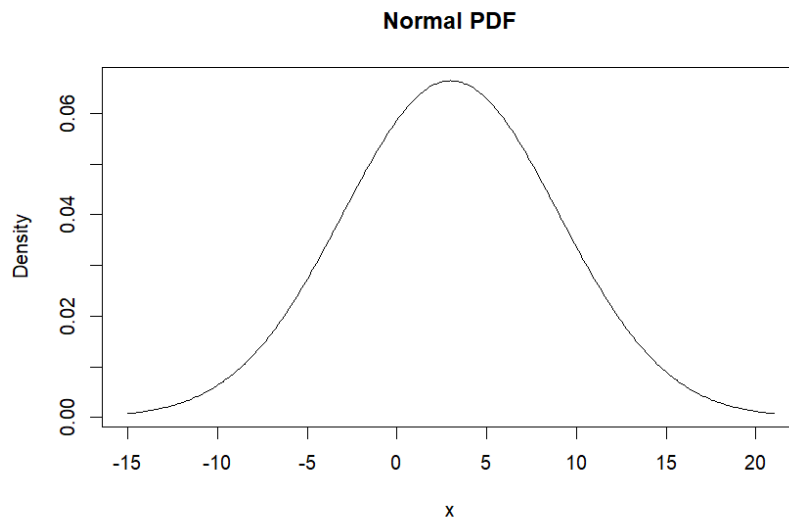
Procedure:

1. Define a continuous range around the mean (-15 to 21).
2. Calculate the PDF using `dnorm()` and CDF using `pnorm()`.

Program:

```
mean <- 3
sd <- 6
x <- seq(-15, 21, by = 0.1)
pdf <- dnorm(x, mean = mean, sd = sd)
plot(x, pdf, type = "l", main = "Normal PDF", xlab = "x", ylab = "Density")
cdf <- pnorm(x, mean = mean, sd = sd)
plot(x, cdf, type = "l", main = "Normal CDF", xlab = "x", ylab = "Cumulative Probability")
```

Output:



9.Problem Statement:

Compute the probability that a random variable from $N(0,1)$ lies between -10 and 8

Aim:

To compute $P(-10 < Z < 8)$ where $Z \sim N(0,1)$ (Standard Normal Distribution).

Procedure:

1. Calculate the difference between the Cumulative probabilities: $CDF(8) - CDF(-10)$ using `pnorm()`.

Program:

```
prob <- pnorm(8, mean = 0, sd = 1) - pnorm(-10, mean = 0, sd = 1)
prob
```

Output:

1

10.Problem Statement:

Calculate the median of an Exponential distribution with rate = 10

Aim:

To find the median of an Exponential distribution with parameter $\lambda = 10$.

Procedure:

1. The median is the point at which the CDF equals 0.5.
2. You can use the quantile function `qexp()` in R, supplying $p = 0.5$.

Program:

```
rate <- 10
median <- qexp(0.5, rate)
median
```

Output:

0.06931472

Signature of the faculty

Ex. No. 5	TESTING OF HYPOTHESIS – Z AND T TESTING
Date : 21.02.2026	

1.Problem Statement:

Perform a one-sample t-test to determine if the mean weight of a sample of 50 individuals is significantly different from 65 kg.

Aim:

To perform a one-sample t-test to test if the sample mean weight is significantly different from the population mean ($\mu = 65$) at a 0.05 significance level.

Procedure:

1. Define the null hypothesis ($H_0: \mu = 65$) and alternative hypothesis ($H_1: \mu \neq 65$).
2. Input the sample data.
3. Conduct the one-sample t-test using the `t.test()` function.
4. Print the test results.
5. Use an if-else statement to evaluate the p.value extracted from the test object. If it is less than 0.05, print the rejection of the null hypothesis; otherwise, fail to reject it.

Program:

```
set.seed(123)
sample_data <- rnorm(50, mean = 68, sd = 5)

mu_val <- 65 # Population parameter (null hypothesis)

t_test <- t.test(sample_data, mu = mu_val) # Conduct one-sample t-test

print(t_test)

if (t_test$p.value < 0.05) {
  cat("Reject null hypothesis: The sample mean is significantly different from", mu_val, "\n")
} else {
  cat("Fail to reject null hypothesis: There is not enough evidence to conclude that the sample mean is different from",
mu_val, "\n")
}
```

Output:

One Sample t-test

```
data: sample_data
t = 4.8451, df = 49, p-value = 1.313e-05
alternative hypothesis: true mean is not equal to 65
95 percent confidence interval:
 66.85637 69.48766
sample estimates:
mean of x
 68.17202
```

Reject null hypothesis: The sample mean is significantly different from 65

2.Problem Statement:

Perform a two-sample t-test to compare the mean scores of two groups: Group A and Group B. Use the following data:

```
Group A <- c(85, 89, 92, 78, 86, 88, 90, 82, 87, 84) group B <- c(80, 81, 85, 79, 83, 81, 84, 78, 82, 80)
```

Aim:

To perform an independent two-sample t-test to determine if there is a significant difference between the mean scores of Group A and Group B.

Procedure:

1. Define the null hypothesis ($H_0: \mu_A = \mu_B$) and alternative hypothesis ($H_1: \mu_A \neq \mu_B$).
2. Define the data arrays for Group A and Group B.
3. Conduct the two-sample t-test using the `t.test()` function.
4. Print the test results.
5. Apply an if-else condition on `p.value` to automatically output whether to accept or reject the null hypothesis.

Program:

```
# Define the data
Group_A <- c(85, 89, 92, 78, 86, 88, 90, 82, 87, 84)
Group_B <- c(80, 81, 85, 79, 83, 81, 84, 78, 82, 80)

# Conduct two-sample t-test
two_sample_t_test <- t.test(Group_A, Group_B)

# Print test result
print(two_sample_t_test)

# Interpretation
if (two_sample_t_test$p.value < 0.05) {
  cat("Reject null hypothesis: There is a significant difference between the means of Group A and Group B.\n")
} else {
  cat("Fail to reject null hypothesis: There is not enough evidence to conclude a difference between the groups.\n")
}
```

Output:

Welch Two Sample t-test

```
data: Group_A and Group_B
t = 3.261, df = 13.847, p-value = 0.005759
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 1.639673 7.960327
sample estimates:
mean of x mean of y
 86.1    81.3
```

Reject null hypothesis: There is a significant difference between the means of Group A and Group B.

3.Problem Statement:

Perform a one-sample Z-test to determine if the mean weight of a sample of 50 individuals is significantly different from 65 kg. Use a significance level of 0.05.

Aim:

To conduct a one-sample Z-test comparing the mean weight of 50 individuals against a population mean of 65 kg, using a known population standard deviation.

Procedure:

1. Define the hypotheses ($H_0: \mu = 65$, $H_1: \mu \neq 65$) and simulate the data.
2. Calculate the sample mean and establish the known population parameters ($\mu = 65$, $\sigma = 5$).
3. Compute the Z-statistic and the corresponding two-tailed p-value mathematically.
4. Print the computed statistics.
5. Use an if-else statement to compare the calculated p-value against the 0.05 threshold to generate the final interpretation.

Program:

```
# Simulate sample data
set.seed(123)
sample_data <- rnorm(50, mean = 68, sd = 5)

# Parameters
mu_val <- 65
sigma <- 5
n <- length(sample_data)
sample_mean <- mean(sample_data)

# Conduct Z-test manually
z_stat <- (sample_mean - mu_val) / (sigma / sqrt(n))
p_value <- 2 * pnorm(-abs(z_stat))

# Print test result
cat("One-Sample Z-Test Results\n")
cat("Z-Statistic:", z_stat, "\nP-value:", p_value, "\n\n")

# Interpretation
if (p_value < 0.05) {
  cat("Reject null hypothesis: The sample mean is significantly different from", mu_val, "\n")
} else {
  cat("Fail to reject null hypothesis: There is not enough evidence to conclude that the sample mean is different from", mu_val, "\n")
}
```

Output:

```
One-Sample Z-Test Results
Z-Statistic: 4.485911
P-value: 7.260321e-06
```

```
Reject null hypothesis: The sample mean is significantly different from 65
```

4.Problem Statement:

Perform a two-sample Z-test to compare the mean scores of two groups: Group A and Group B. Use the following data: # Example data

Group A <- c(85, 89, 92, 78, 86, 88, 90, 82, 87, 84) group B <- c(80, 81, 85, 79, 83, 81, 84, 78, 82, 80)

Aim:

To execute a two-sample Z-test to find out if the means of Group A and Group B differ significantly.

Procedure:

1. Define the hypotheses ($H_0: \mu_A = \mu_B$, $H_1: \mu_A \neq \mu_B$) and input the data for both groups.
2. Calculate the means, variances, and lengths of both sample groups.
3. Compute the two-sample Z-statistic and extract the two-tailed p-value.
4. Print the computed test statistics.
5. Implement an if-else logical block to automatically interpret the generated p-value.

Program:

```
# Define the data
Group_A <- c(85, 89, 92, 78, 86, 88, 90, 82, 87, 84)
Group_B <- c(80, 81, 85, 79, 83, 81, 84, 78, 82, 80)

# Calculate statistics for the test
mean_A <- mean(Group_A); var_A <- var(Group_A); n_A <- length(Group_A)
mean_B <- mean(Group_B); var_B <- var(Group_B); n_B <- length(Group_B)

# Conduct two-sample Z-test manually
z_stat_2 <- (mean_A - mean_B) / sqrt((var_A / n_A) + (var_B / n_B))
p_value_2 <- 2 * pnorm(-abs(z_stat_2))

# Print test result
cat("Two-Sample Z-Test Results\n")
cat("Z-Statistic:", z_stat_2, "\nP-value:", p_value_2, "\n\n")

# Interpretation
if (p_value_2 < 0.05) {
  cat("Reject null hypothesis: There is a significant difference between the means of Group A and Group B.\n")
} else {
  cat("Fail to reject null hypothesis: There is not enough evidence to conclude a difference between the groups.\n")
}
```

Output:

```
Two-Sample Z-Test Results
Z-Statistic: 3.260958
P-value: 0.001110365
```

Reject null hypothesis: There is a significant difference between the means of Group A and Group B.

Signature of the faculty

Ex. No. 6	TESTING OF HYPOTHESIS – F AND CHI-SQUARE TESTING
Date : 07.03.2026	

1(A).Problem Statement:

Compare variance of Group A and Group B

Aim:

To check whether both groups have equal variance.

Procedure:

1. Calculate variance of Group A and Group B
2. Compute $F = (\text{larger variance} / \text{smaller variance})$
3. Compare with critical value or p-value).

Program:

```
groupA <- c(72, 75, 78, 71, 74, 77, 76, 73, 75, 78)
groupB <- c(68, 71, 73, 69, 72, 70, 72, 67, 71, 74)
f_test1 <- var.test(groupA, groupB)
print(f_test1)
if (f_test1$p.value < 0.05) {
  cat("Reject null hypothesis: Variances are different\n")
} else {
  cat("Fail to reject null hypothesis: Variances are same\n")
}
```

Output:

F test to compare two variances

```
data: groupA and groupB
F = 1.1995, num df = 9, denom df = 9, p-value = 0.7908
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
0.2979504 4.8293671
sample estimates:
ratio of variances
1.199546
Fail to reject null hypothesis: Variances are same
```

1(B).Problem Statement:

Compare variance of Sample A and Sample B.

Aim:

To determine if variability differs between samples.

Procedure:

1. Find mean and variance of both samples
2. Calculate F-ratio
3. Interpret using F-table or p-value

Program:

```
sampleA <- c(8, 9, 6, 7, 10)
sampleB <- c(12, 15, 14, 13, 16)

f_test2 <- var.test(sampleA, sampleB)

print(f_test2)

if (f_test2$p.value < 0.05) {
  cat("Reject null hypothesis: Variances are different\n")
} else {
  cat("Fail to reject null hypothesis: Variances are same\n")
}
```

Output:

F test to compare two variances

data: sampleA and sampleB
F = 1, num df = 4, denom df = 4, p-value = 1
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
0.1041175 9.6045299
sample estimates:
ratio of variances
1

Fail to reject null hypothesis: Variances are same

1(C).Problem Statement:

Test variance difference between Sample A and B.

Aim:

To analyze equality of variances.

Procedure:

1. Compute sample variances
2. Form F-statistic
3. Compare with critical value

Program:

```
sampleA <- c(20, 22, 19, 24, 21)
sampleB <- c(30, 35, 28, 32, 31)

f_test3 <- var.test(sampleA, sampleB)

print(f_test3)

if (f_test3$p.value < 0.05) {
  cat("Reject null hypothesis: Variances are different\n")
} else {
  cat("Fail to reject null hypothesis: Variances are same\n")
}
```

Output:

```
F test to compare two variances

data: sampleA and sampleB
F = 0.55224, num df = 4, denom df = 4, p-value = 0.5793
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.05749774 5.30399412
sample estimates:
ratio of variances
 0.5522388
Fail to reject null hypothesis: Variances are same
```

2(A).Problem Statement:

Test association between gender and drink preference.

Aim:

To check if variables are independent.

Procedure:

1. Calculate expected frequencies
2. Compute χ^2 value
3. Compare with critical value / p-value

Program:

```
observed1 <- matrix(c(40, 30,
  20, 50),
  nrow = 2,
  byrow = TRUE)

chi1 <- chisq.test(observed1)

print(chi1)

if (chi1$p.value < 0.05) {
  cat("Reject null hypothesis: Association exists\n")
} else {
  cat("Fail to reject null hypothesis: No association\n")
}
```

Output:

Pearson's Chi-squared test with Yates' continuity correction

data: observed1

X-squared = 10.529, df = 1, p-value = 0.001175

Reject null hypothesis: Association exists

2(B).Problem Statement:

Compare observed and expected frequencies.

Aim:

To check goodness of fit.

Procedure:

1. Use given expected frequencies
2. Calculate χ^2 statistic
3. Interpret result

Program:

```
observed2 <- c(25, 30, 20, 25)
```

```
expected2 <- c(25, 25, 25, 25)
```

```
chi2 <- chisq.test(observed2, p = expected2/sum(expected2))
```

```
print(chi2)
```

```
if (chi2$p.value < 0.05) {  
  cat("Reject null hypothesis: Not equal distribution\n")  
} else {  
  cat("Fail to reject null hypothesis: Equal distribution\n")  
}
```

Output:

Chi-squared test for given probabilities

data: observed2

X-squared = 2, df = 3, p-value = 0.5724

Fail to reject null hypothesis: Equal distribution

3.Problem Statement:

Compare the variances of two groups (group1 and group2).

Aim:

To determine whether the variances of the two groups are equal.

Procedure:

1. State hypotheses: H_0 (variances are equal), H_1 (variances are not equal)
2. Perform F-test using given data
3. Compare p-value with significance level (0.05)

Program:

```
data(mtcars)

group1 <- mtcars$mpg[mtcars$cyl == 4]
group2 <- mtcars$mpg[mtcars$cyl == 6]

f_test_real <- var.test(group1, group2)

print(f_test_real)

if (f_test_real$p.value < 0.05) {
  cat("Reject null hypothesis\n")
} else {
  cat("Fail to reject null hypothesis\n")
}
```

Output:

```
F test to compare two variances

data: group1 and group2
F = 9.6261, num df = 10, denom df = 6, p-value = 0.01182
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 1.762592 39.198688
sample estimates:
ratio of variances
 9.626086
Reject null hypothesis
```

Signature of the faculty

Ex. No. 07	MARKOV CHAINS ANALYSIS USING MARKOVCHAIN PACKAGE IN R
Date : 14.03.2026	

1.Problem Statement:

Model disease progression between Healthy, Infected, and Recovered states.

Aim:

To analyze long-term behavior and simulate disease spread.

Procedure:

1. Define transition matrix and create Markov chain
2. Compute steady-state probabilities
3. Simulate progression for 15 steps

Program:

```
disease_matrix <- matrix(c(0.85, 0.10, 0.05, 0.10, 0.70, 0.20, 0.00, 0.05, 0.95),
  byrow = TRUE, nrow = 3, dimnames = list(c("Healthy", "Infected", "Recovered"),
  c("Healthy", "Infected", "Recovered")))

disease_chain <- new("markovchain", states = c("Healthy", "Infected", "Recovered"),
  transitionMatrix = disease_matrix)

steady_state <- steadyStates(disease_chain)

print("Steady-State Probabilities:")

print(steady_state)

set.seed(123)

simulation <- rmarkovchain(n = 15, object = disease_chain, t0 = "Healthy")

print("Disease Progression Simulation:")

print(simulation)
```

Output:

```
[1] "Steady-State Probabilities:"
Healthy Infected Recovered
[1,] 0.1052632 0.1578947 0.7368421

[1] "Disease Progression Simulation:"
[1] "Healthy" "Healthy" "Healthy" "Infected" "Healthy" "Healthy" "Healthy"
[8] "Infected" "Infected" "Infected" "Healthy" "Healthy" "Healthy" "Healthy"
[15] "Healthy"
```

2.Problem Statement:

Analyze user movement between web pages A, B, and C.

Aim:

To compute PageRank using steady-state probabilities.

Procedure:

1. Define transition matrix
2. Calculate steady-state probabilities
3. Simulate navigation for 20 steps

Program:

```
pagerank_matrix <- matrix(
  c(0.1, 0.6, 0.3,
    0.3, 0.4, 0.3,
    0.4, 0.2, 0.4),
  byrow = TRUE,
  nrow = 3,
  dimnames = list(c("A", "B", "C"), c("A", "B", "C")))
pagerank_chain <- new("markovchain", states = c("A", "B", "C"), transitionMatrix = pagerank_matrix)
pagerank <- steadyStates(pagerank_chain)
print("PageRank Steady-State Probabilities:")
print(pagerank)
set.seed(123)
navigation <- rmarkovchain(n = 20, object = pagerank_chain, t0 = "A")
print("Simulated User Navigation:")
print(navigation)
```

Output:

```
[1] "PageRank Steady-State Probabilities:"
      A      B      C
[1,] 0.2777778 0.3888889 0.3333333

[1] "Simulated User Navigation:"
[1] "B" "C" "C" "B" "C" "A" "B" "C" "C" "C" "B" "A" "C" "C" "A" "C" "A" "B" "B" "C"
```

3.Problem Statement:

Model stock market states (Bull, Bear, Stagnant).

Aim:

To predict future market behavior and probabilities.

Procedure:

1. Define Markov chain transition matrix
2. Simulate behavior for 12 steps
3. Compute probability after 6 steps

Program:

```
stock_market_matrix <- matrix(c(0.7, 0.2, 0.1, 0.4, 0.4, 0.2, 0.3, 0.3, 0.4),
  byrow = TRUE, nrow = 3, dimnames = list(c("Bull", "Bear", "Stagnant"), c("Bull", "Bear", "Stagnant")))
stock_market_chain <- new("markovchain", states = c("Bull", "Bear", "Stagnant"),
  transitionMatrix = stock_market_matrix)

set.seed(123)
simulation <- rmarkovchain(n = 12, object = stock_market_chain, t0 = "Bull")
print("Stock Market Behavior Simulation:")
print(simulation)
six_step_matrix <- stock_market_chain^6
print("Transition Matrix After 6 Steps:")
print(six_step_matrix)
```

Output:

```
[1] "Stock Market Behavior Simulation:"
[1] "Bull"  "Bear"  "Bear"  "Stagnant" "Bear"  "Bull"  "Bull"
[8] "Bear"  "Bear"  "Bear"  "Stagnant" "Bull"

[1] "Transition Matrix After 6 Steps:"
      Bull  Bear Stagnant
Bull  0.546375 0.272375 0.181250
Bear  0.544750 0.273000 0.182250
Stagnant 0.543750 0.273375 0.182875
```

4.Problem Statement:

Analyze transitions between Free, Trial, and Jail states.

Aim:

To identify absorbing state and study system behavior.

Procedure:

1. Define transition matrix
2. Identify absorbing states
3. Simulate process for 10 steps

Program:

```
prisoner_matrix <- matrix(c(0.5, 0.4, 0.1, 0.3, 0.5, 0.2, 0.0, 0.0, 1.0), byrow = TRUE,
  nrow = 3, dimnames = list(c("Free", "Trial", "Jail"), c("Free", "Trial", "Jail")))
prisoner_chain <- new("markovchain", states = c("Free", "Trial", "Jail"),
  transitionMatrix = prisoner_matrix)
absorbing <- absorbingStates(prisoner_chain)
print("Absorbing States:")
print(absorbing)
set.seed(123)
simulation <- rmarkovchain(n = 10, object = prisoner_chain, t0 = "Free")
print("Prisoner Dilemma Simulation:")
print(simulation)
```

Output:

```
[1] "Absorbing States:"
[1] "Jail"

[1] "Prisoner Dilemma Simulation:"
[1] "Free" "Trial" "Trial" "Jail" "Jail" "Jail" "Jail" "Jail" "Jail" "Jail"
```

5.Problem Statement:

Model production states (Good, Defective, Repaired).

Aim:

To analyze long-term production quality.

Procedure:

1. Define Markov chain
2. Compute steady-state probabilities
3. Simulate process for 20 steps

Program:

```
manufacturing_matrix <- matrix(c(0.9, 0.1, 0.0, 0.3, 0.5, 0.2, 0.8, 0.1, 0.1),
  byrow = TRUE, nrow = 3, dimnames = list(c("Good", "Defective", "Repaired"),
  c("Good", "Defective", "Repaired")))
manufacturing_chain <- new("markovchain", states = c("Good", "Defective",
  "Repaired"),
  transitionMatrix = manufacturing_matrix)
steady_state <- steadyStates(manufacturing_chain)
print("Steady-State Probabilities:")
```

```
print(steady_state)
set.seed(123)
simulation <- rmarkovchain(n=20, object= manufacturing_chain, t0 ="Defective")
print("Manufacturing Process Simulation:")
print(simulation)
```

Output:

```
[1] "Steady-State Probabilities:"
      Good Defective Repaired
[1,] 0.7962963 0.1666667 0.03703704

[1] "Manufacturing Process Simulation:"
[1] "Defective" "Good"    "Good"    "Good"    "Defective" "Defective"
[7] "Good"     "Good"     "Good"     "Good"     "Defective" "Defective"
[13] "Good"     "Good"     "Good"     "Good"     "Good"      "Good"
[19] "Good"     "Defective"
```

Signature of the faculty

Ex. No. 8

Date :
28.03.2026

QUEUEING MODEL ANALYSIS USING QUEUEING PACKAGE IN R

1.Problem Statement:

Model an M/M/1 system with:

- Arrival rate ($\lambda=4$ \lambda=4)
- Service rate ($\mu=6$ \mu=6).

Aim:

To model a single-server queueing system (M/M/1) and compute key performance measures such as waiting time and server utilization.

Procedure:

1. Define the input parameters using NewInput.MM1.
2. Compute the model using QueueingModel.
3. Extract key performance measures.

Program:

```
library(queueing)
mm1_input <- NewInput.MM1(lambda = 4, mu = 6)
mm1_model <- QueueingModel(mm1_input)
summary(mm1_model)
```

Output:

lambda	mu	c	k	m	RO	P0	Lq	Wq	X	L	W	Wqq	Lqq
4	6	1	NA	NA	0.6666667	0.3333333	1.333333	0.3333333	4	2	0.5	0.5	3

2.Problem Statement:

Model an M/M/3 system with:

- $\lambda=10$ \lambda=10
- $\mu=5$ \mu=5
- $c=3$

Aim:

To model a multi-server queueing system (M/M/c) and calculate key performance metrics (such as waiting time and system length) for a system with parallel processing servers.

Procedure:

1. Define the input parameters for the M/M/c system using the NewInput.MMC function, specifying lambda, mu, and c.
2. Compute the queueing model using the QueueingModel() function.
3. Extract and display key performance measures using the summary() function.

Program:

```
library(queueing)
mmc_input <- NewInput.MMC(lambda = 10, mu = 5, c = 3)
mmc_model <- QueueingModel(mmc_input)
summary(mmc_model)
```

Output:

lambda	mu	c	k	m	RO	P0	Lq	Wq	X	L	W	Wqq	Lqq
10	5	3	NA	NA	0.6666667	0.11111111	0.88888889	0.08888889	10	2.888889	0.2888889	0.2	3

3.Problem Statement:

Model an M/M/1/K system with:

- $\lambda=8 \backslash \lambda=8 \lambda=8$,
- $\mu=12 \backslash \mu=12 \mu=12$
- $K=5 K$.

Aim:

To model a finite capacity M/M/1/K queuing system and determine key performance metrics such as the blocking probability and average system length.

Procedure:

- 1 Define the input parameters for the M/M/1/K system using the NewInput.MM1K function.
- 2 Compute the queuing model using the QueueingModel() function.
- 3 Extract and display the performance measures using the summary() function.

Program:

```
mm1k_input <- NewInput.MM1K(lambda = 8, mu = 12, k = 5)
mm1k_model <- QueueingModel(mm1k_input)
summary(mm1k_model)
```

Output:

Lambda	mu	c	k	m	RO	P0	Lq	Wq	X	L	W
8	12	1	5	NA	0.6345865	0.3654135	0.7879699	0.1034755	7.615038	1.422556	0.1868088
					Wqq		Lqq				
					0.1679487		2.015385				

4.Problem Statement:

Model an M/M/2/4 system with:

- $\lambda=4 \backslash \lambda=4 \lambda=4$
- $\mu=5 \backslash \mu=5 \mu=5$
- $c=2 \backslash c=2 c=2$
- $K=4 \backslash K=4 K=4$.

Aim:

To model a multi-server queuing system with finite capacity (M/M/c/K) and determine performance metrics such as the blocking probability and average system length.

Procedure:

1. Define the input parameters for the M/M/2/4 system using the NewInput.MMCK function.
2. Compute the queuing model using the QueueingModel() function.
3. Extract and display the performance measures using the summary() function.

Program:

```
mmc_k_input <- NewInput.MMCK(lambda = 4, mu = 5, c = 2, k = 4)
mmc_k_model <- QueueingModel(mmc_k_input)
summary(mmc_k_model)
```

Output:

```
lambda mu c k m      RO      PO      Lq      Wq      X      L      W
4 5 2 4 NA 0.3910926 0.4349339 0.1002088 0.02562278 3.910926 0.8823939 0.2256228
      Wqq      Lqq
0.1285714 1.285714
```

5.Problem Statement:

Model a priority queue with two customer types:

- High-priority ($\lambda_1=3$ \lambda_1=3)
- Low-priority ($\lambda_2=2$ \lambda_2=2)
- Service rate ($\mu=5$ \mu=5).

Aim:

To model a priority queuing system that differentiates between multiple customer classes and calculates specific performance metrics for each priority level.

Procedure:

1. Define the input parameters for the priority queue using the NewInput.PM function, providing λ as a vector and the priority levels.
2. Compute the queuing model using the QueueingModel() function.
3. Extract and display the performance measures using the summary() function.

Program:

```
priority_input <- NewInput.PM(lambda = c(3, 2), mu = 5, prio = c(1, 2))
priority_model <- QueueingModel(priority_input)
summary(priority_model)
```

Output:

```
Priority Queueing Model Summary
Metric Class_1 Class_2 Total
1      Lq      1.5      Inf      Inf
2      Wq      0.5      Inf      Inf
3       L      2.1      Inf      Inf
4       W      0.7      Inf      Inf
5      Rho      0.6      0.4      1
```

6.Problem Statement:

Model an open network with:

- Node 1: M/M/1 ($\mu=4$)
- Node 2: M/M/1 ($\mu=6$).

Transition probabilities:

- P1 to 2 = 0.6
- P2 to 1 = 0.4.

Aim:

To model an open queuing network and analyze the performance of individual nodes when traffic flows between them.

Procedure:

1. Define the nodes as a list of Node.MM1 objects, specifying arrival (λ) and service (μ) rates.
2. Define the transition matrix to represent the routing probabilities between nodes.
3. Use NewInput.Open to integrate the nodes and the transition matrix.
4. Compute the queuing model using QueueingModel() and extract performance measures using summary().

Program:

```
library(queueing)
nodes <- list(
  Node.MM1(lambda = 4, mu = 4),
  Node.MM1(lambda = 4, mu = 6)
)
trans_matrix <- matrix(c(0, 0.6, 0.4, 0), nrow = 2, byrow = TRUE)
open_net <- NewInput.Open(nodes, prob = trans_matrix)
open_model <- QueueingModel(open_net)
summary(open_model)
```

Output:

	L	W	X		Lk	Wk	Xk	ROk
	4.666667	1.166667	4		NA	NA	NA	NA
	NA		NA NA	4.000000	1.000000	4.0	0.8	
	NA		NA NA	0.666667	0.166667	2.4	0.4	

7.Problem Statement:

Model a closed network with:

- 3 nodes: Node 1 ($\mu=5$)
- Node 2 ($\mu=4$)
- Node 3 ($\mu=6$)
- 10 customers circulating.

Aim:

To model a closed queuing network where a fixed number of customers circulate among the nodes, and to analyze performance metrics like utilization and throughput under these constrained conditions.

Procedure:

1. Define the network nodes as a list of Node.MM1 objects.
2. Set the input parameters using NewInput.Closed, specifying the nodes and the total number of customers.
3. Compute the model using QueueingModel() and display the summary.

Program:

```
nodes_closed <- list(
  Node.MM1(lambda = 0, mu = 5),
  Node.MM1(lambda = 0, mu = 4),
  Node.MM1(lambda = 0, mu = 6)
)
closed_net_input <- NewInput.Closed(nodes = nodes_closed, customers = 10)
closed_net_model <- QueueingModel(closed_net_input)
summary(closed_net_model)
```

Output:

L	W	X	Lk	Wk	Xk	ROk
10.000000	3.421053	2.923077	NA	NA	NA	NA
NA	NA	NA	4.153846	1.421053	2.923077	0.584615
NA	NA	NA	3.230769	1.105263	2.923077	0.730769
NA	NA	NA	2.615385	0.894737	2.923077	0.487179

8.Problem Statement:

Model a production line with three stages:

- Stage 1 ($\mu=10$ \mu = 10 $\mu=10$)
- Stage 2 ($\mu=8$ \mu = 8 $\mu=8$)
- Stage 3 ($\mu=12$ \mu = 12 $\mu=12$).

Aim:

To model a multi-stage production line using an open queuing network to analyze the throughput and identify potential bottlenecks in the system.

Procedure:

1. Create a list of Node.MM1 objects representing the three stages of the production line.
2. Define the open queuing network input using NewInput.Open.
3. Compute the queuing model using QueueingModel() and summarize the results.

Program:

```
prod_line <- list(
  Node.MM1(lambda = 7, mu = 10),
  Node.MM1(lambda = 7, mu = 8),
  Node.MM1(lambda = 7, mu = 12)
)
prod_input <- NewInput.Open(nodes = prod_line)
prod_model <- QueueingModel(prod_input)
summary(prod_model)
```

Output:

```
Net 10.73333 1.533333 7 NA NA NA NA
Nd1 NA NA NA 2.333333 0.333333 7 0.7000000
Nd2 NA NA NA 7.000000 1.000000 7 0.8750000
Nd3 NA NA NA 1.400000 0.200000 7 0.5833333
> |
```

9.Problem Statement:

Model an M/E2/1 queue with:

- Arrival rate ($\lambda=3$ \lambda = 3)
- Service rate ($\mu=6$ \mu = 6).

Aim:

To model an M/Ek/1 queuing system (Erlang distribution service time) and analyze the system's performance metrics.

Procedure:

1. Define the input parameters for the M/E2/1 system
2. Compute the model using QueueingModel().
3. Extract and display performance measures using summary().

Program:

```
erlang_input <- NewInput.MM1(lambda = 3, mu = 6)
erlang_model <- QueueingModel(erlang_input)
summary(erlang_model)
```

Output:

```
lambda mu c k m RO P0 Lq Wq X L W Wq Lqq
3 6 1 NA NA 0.5 0.5 0.5 0.1666667 3 1 0.3333333 0.3333333 2
```

10.Problem Statement:

Model a tandem queuing system with two queues:

- Queue 1 ($\mu=5$ \mu = 5)
- Queue 2 ($\mu=7$ \mu = 7)
- Arrival rate ($\lambda=4$ \lambda = 4).

Aim:

To model a tandem queuing system (queues in series) and analyze the performance metrics of the entire system as customers flow from the first node to the second.

Procedure:

1. Define the tandem queues as a list of Node.MM1 objects with their respective service rates.
2. Create the network input using NewInput.Open to link the nodes in sequence.
3. Compute the model using QueueingModel().
4. Extract and display the performance measures using summary().

Program:

```
tandem_queues <- list(  
  Node.MM1(lambda = 4, mu = 5),  
  Node.MM1(lambda = 4, mu = 7)  
)  
tandem_input <- NewInput.Open(nodes = tandem_queues)  
tandem_model <- QueueingModel(tandem_input)  
summary(tandem_model)
```

Output:

	L	W	X	Lk	wk	Xk	R0k
Net	5.333333	1.333333	4	NA	NA	NA	NA
Nd1	NA	NA	NA	4.000000	1.000000	4	0.800000
Nd2	NA	NA	NA	1.333333	0.333333	4	0.5714286

Signature of the faculty

Ex. No. 9	MONTE CARLO SIMULATION - PREDICTING STOCK PRICES USING MONTECARLO PACKAGE IN R
Date : 04.04.2026	

1.Problem Statement:

Simulate a single stock's price for 1 year with:

- Initial price ($S_0=100, S_0 = 100, S_0=100$)
- Daily return mean ($\mu=0.001, \mu = 0.001, \mu=0.001$)
- Volatility ($\sigma=0.02, \sigma = 0.02, \sigma=0.02$)
- 252 trading days.

Aim:

To Simulate a Single Stock Price.

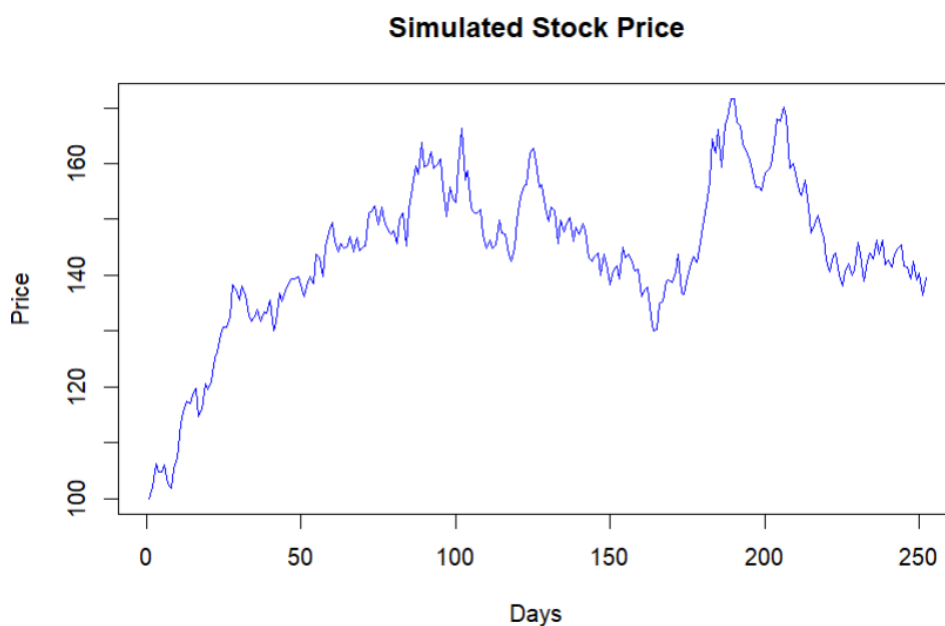
Procedure:

1. Define a function to simulate daily prices.
2. Run the simulation.
3. Plot the price trajectory.

Program:

```
S0 <- 100
mu <- 0.001
sigma <- 0.02
T <- 252
n_sims <- 1000
simulate_stock <- function(S0, mu, sigma, T){
  prices <- numeric(T)
  prices[1] <- S0
  for (t in 2:T){
    prices[t] <- prices[t - 1] * exp(rnorm(1, mean = mu, sd = sigma))
  }
  return(prices)
}
prices <- simulate_stock(S0, mu, sigma, T)
plot(prices, type = "l", col = "blue", main = "Simulated Stock Price", xlab = "Days", ylab = "Price")
```

Output:



2.Problem Statement:

Simulate a multiple stock's price for 1 year with:

- Initial price ($S_0=100, S_0 = 100, S_0=100$)
- Daily return mean ($\mu=0.001, \mu = 0.001, \mu=0.001$)
- Volatility ($\sigma=0.02, \sigma = 0.02, \sigma=0.02$)
- 252 trading days.

Aim:

To simulate and visualize multiple potential stock price trajectories simultaneously, allowing for the comparison of different outcomes generated by the same stochastic parameters (μ and σ).

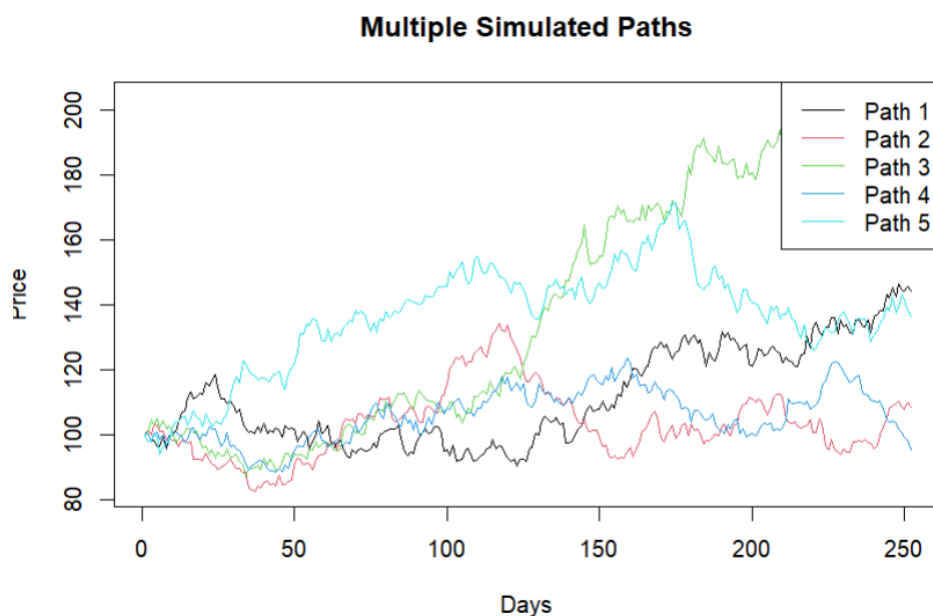
Procedure:

1. Modify the simulation function to handle multiple simulations.
2. Execute the function to generate a matrix of price paths.
3. Use matplotlib to overlay these paths on a single graph for comparative analysis.

Program:

```
S0 <- 100
mu <- 0.001
sigma <- 0.02
T <- 252
n_sims <- 1000
simulate_multiple <- function(S0, mu, sigma, T, n_paths){
  paths <- matrix(NA, nrow = T, ncol = n_paths)
  for (i in 1:n_paths){
    paths[, i] <- simulate_stock(S0, mu, sigma, T)
  }
  return(paths)
}
n_paths <- 5
paths <- simulate_multiple(S0, mu, sigma, T, n_paths)
matplot(paths, type = "l", lty = 1, col = 1:n_paths, main = "Multiple Simulated Paths", xlab = "Days", ylab = "Price")
legend("topright", legend = paste("Path", 1:n_paths), col = 1:n_paths, lty = 1)
```

Output:



3.Problem Statement:

- Initial price ($S_0=100, S_0 = 100, S_0=100$)
- Daily return mean ($\mu=0.001, \mu = 0.001, \mu=0.001$)
- Volatility ($\sigma=0.02, \sigma = 0.02, \sigma=0.02$)
- 252 trading days.

Aim:

To estimate the stock price at the end of 1 year using 1,000 simulations.

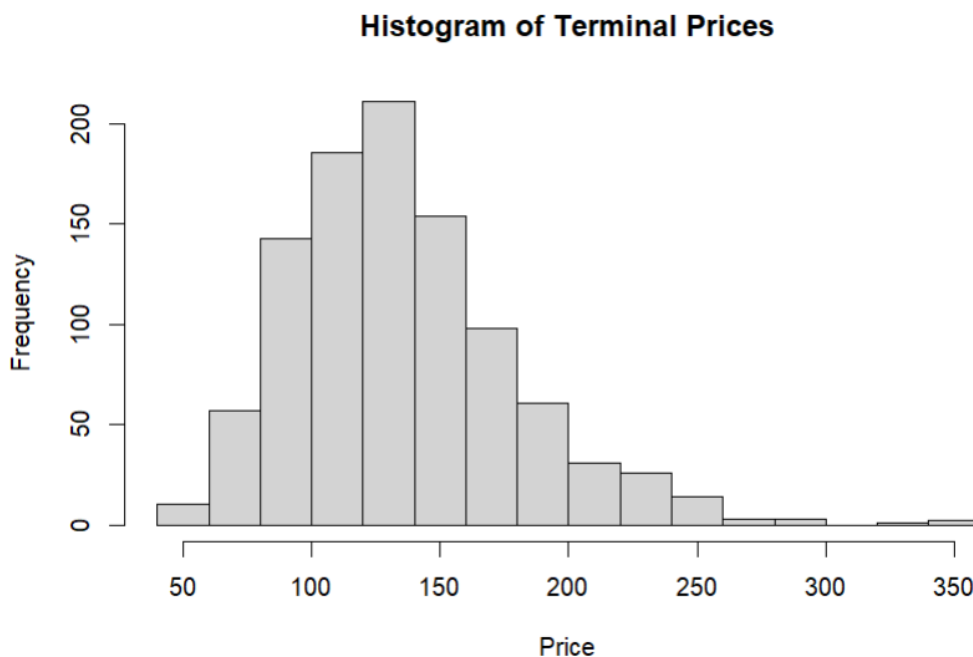
Procedure:

1. Simulate terminal prices.
2. Calculate the mean and variance

Program:

```
S0 <- 100
mu <- 0.001
sigma <- 0.02
T <- 252
n_sims <- 1000)
simulate_terminal <- function(S0, mu, sigma, T, n_sims) {
  terminal_prices <- numeric(n_sims)
  for (i in 1:n_sims) {
    prices <- simulate_stock(S0, mu, sigma, T)
    terminal_prices[i] <- prices[T]
  }
  return(terminal_prices)
}
terminal_prices <- simulate_terminal(S0, mu, sigma, T, n_sims)
mean(terminal_prices)
var(terminal_prices)
hist(terminal_prices, main = "Histogram of Terminal Prices", xlab = "Price")
```

Output:



4.Problem Statement:

Simulate a single stock's price for 1 year with:

- Initial price ($S_0=100, S_{-0} = 100, S_0=100$)
- Daily return mean ($\mu=0.001, \mu = 0.001, \mu=0.001$)
- Volatility ($\sigma=0.02, \sigma = 0.02, \sigma=0.02$)
- 252 trading days.

Aim:

To implement a Monte Carlo simulation model for forecasting stock price trajectories by incorporating a deterministic drift component alongside stochastic volatility, as defined by the equation:

$$\text{Price}(t) = \text{Price}(t-1) * e^{(\mu * \Delta t + \sigma * Z)}$$

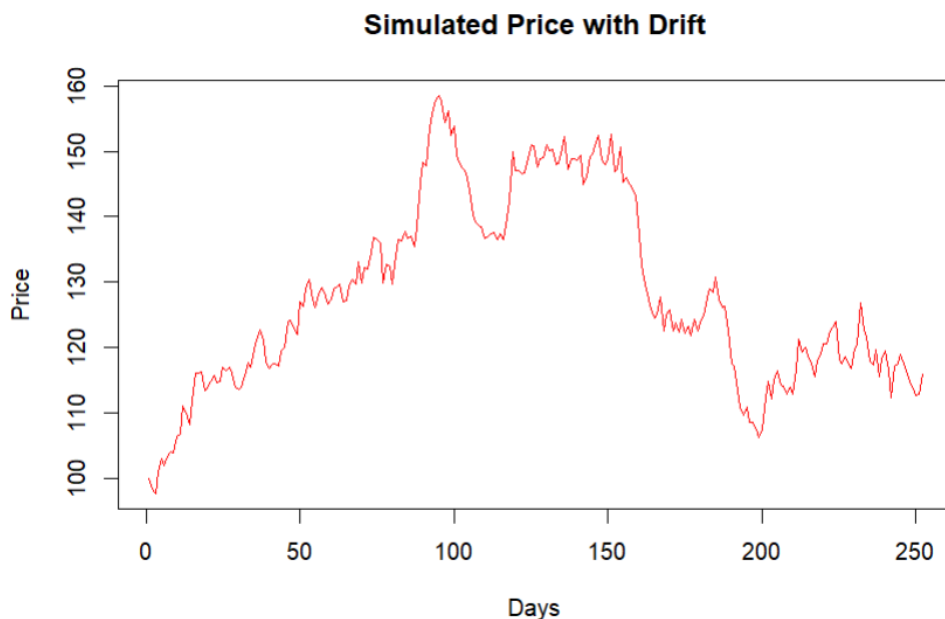
Procedure:

1. Define the simulation function with an added drift component.
2. Initialize the required market parameters.
3. Execute the code to generate and visualize the stock price trajectory.

Program:

```
S0 <- 100
mu <- 0.001
sigma <- 0.02
T <- 252
n_sims <- 1000
simulate_with_drift <- function(S0, mu, sigma, T, drift = 0.0005) {
  prices <- numeric(T)
  prices[1] <- S0
  for (t in 2:T) {
    prices[t] <- prices[t - 1] * exp(mu + drift + sigma * rnorm(1))
  }
  return(prices)
}
prices_drift <- simulate_with_drift(S0, mu, sigma, T)
plot(prices_drift, type = "l", col = "red", main = "Simulated Price with Drift", xlab = "Days", ylab = "Price")
```

Output:



5.Problem Statement:

Simulate daily log returns and analyze their distribution.

Aim:

To simulate daily log returns using a normal distribution and perform statistical analysis (mean and standard deviation) to verify they approximate the intended input parameters (mu and sigma)

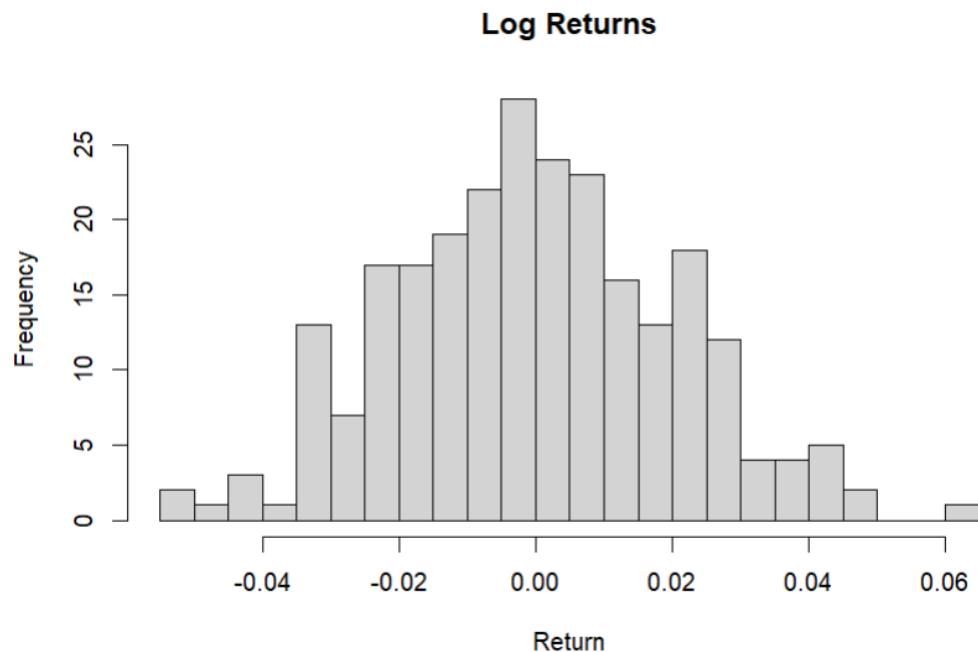
Procedure:

1. Define a function to generate random log returns.
2. Simulate the dataset over the specified period.
3. Visualize the distribution using a histogram while calculating the mean and standard deviation.

Program:

```
S0 <- 100
mu <- 0.001
sigma <- 0.02
T <- 252
n_sims <- 1000
simulate_log_returns <- function(mu, sigma, T) {
  mnorm(T, mean = mu, sd = sigma)
}
log_returns <- simulate_log_returns(mu, sigma, T)
hist(log_returns, breaks = 30, main = "Log Returns", xlab = "Return")
```

Output:



6.Problem Statement:

Estimate the 5% Value-at-Risk (VaR) for a portfolio using Monte Carlo simulation.

Aim:

To quantify financial risk by estimating the 5% Value-at-Risk (VaR) for an investment portfolio using Monte Carlo simulation techniques.

Procedure:

1. Define a function to simulate the terminal prices of an asset.
2. Execute the simulation across a large number of trials.
3. Compute the 5th percentile (quantile) of the resulting distribution to determine the potential loss threshold.

Program:

```
S0 <- 100
mu <- 0.001
sigma <- 0.02
T <- 252
n_sims <- 1000
VaR <- function(S0, mu, sigma, T, n_sims, alpha = 0.05) {
  terminal_prices <- simulate_terminal(S0, mu, sigma, T, n_sims)
  quantile(terminal_prices, alpha)
}
VaR(S0, mu, sigma, T, n_sims)
```

Output:

```
5%
72.38072
```

7.Problem Statement:

Evaluate the impact of varying levels of volatility (sigma) on the terminal prices of an asset.

Aim:

To perform a sensitivity analysis on the Monte Carlo simulation model to understand how changes in input volatility affect the expected (mean) terminal price of the asset.

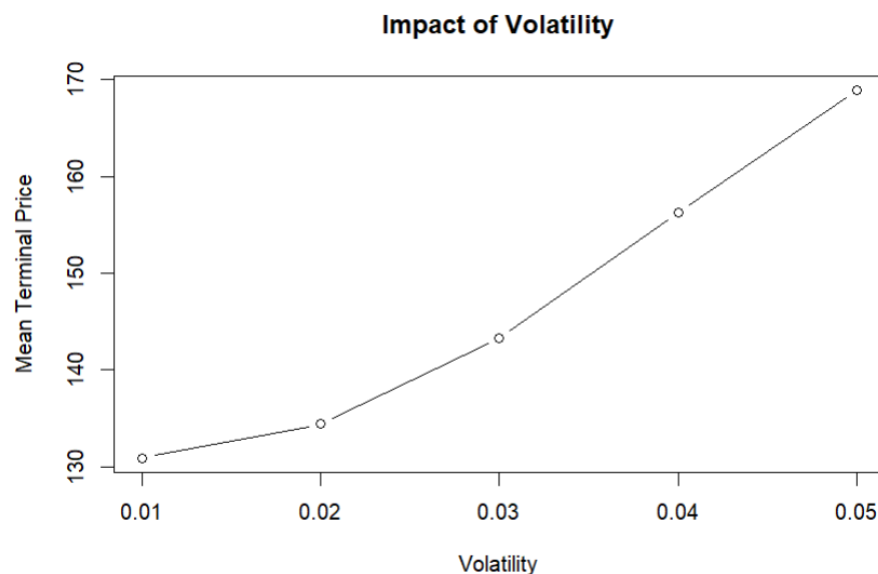
Procedure:

1. Define a range of volatility values.
2. Execute the terminal price simulation for each value in that range.
3. Generate a plot to visualize the correlation between volatility and the mean terminal price.

Program:

```
S0 <- 100
mu <- 0.001
sigma <- 0.02
T <- 252
n_sims <- 1000
volatilities <- seq(0.01, 0.05, by = 0.01)
results <- sapply(volatilities, function(sigma_val) mean(simulate_terminal(S0, mu, sigma_val, T, n_sims)))
plot(volatilities, results, type = "b", main = "Impact of Volatility", xlab = "Volatility", ylab = "Mean Terminal Price")
```

Output:



8.Problem Statement:

Predict the distribution of stock prices after 1 year.

Aim:

To forecast the potential range and probability distribution of future stock prices after one year by generating a histogram of terminal prices derived from a Monte Carlo simulation.

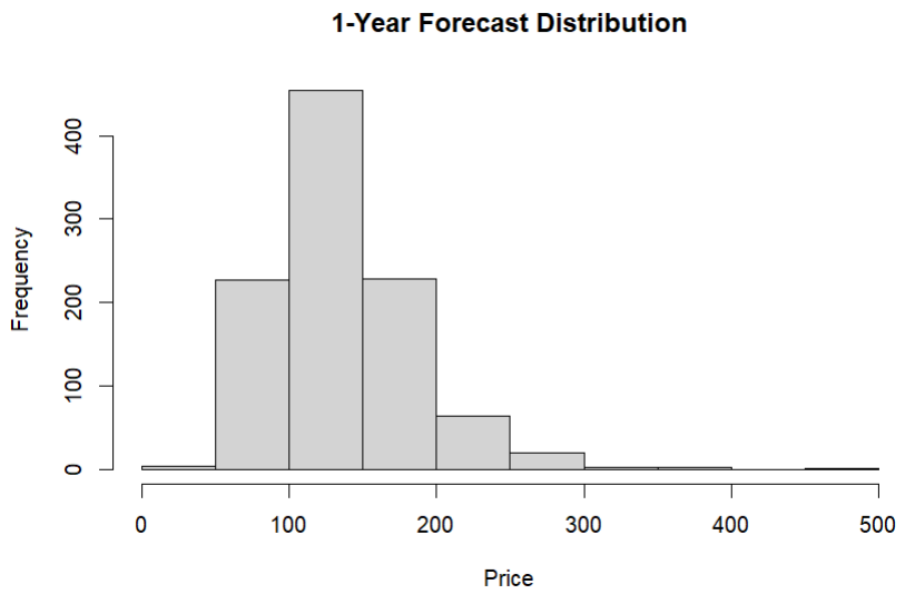
Procedure:

1. Simulate 1,000 terminal stock prices using the `simulate_terminal` function.
2. Plot the results as a histogram to visualize the distribution of forecasted prices.

Program:

```
S0 <- 100
mu <- 0.001
sigma <- 0.02
T <- 252
n_sims <- 1000
forecast_prices <- simulate_terminal(S0, mu, sigma, T, n_sims)
hist(forecast_prices, main = "1-Year Forecast Distribution", xlab = "Price")
```

Output:



9.Problem Statement:

Use Monte Carlo simulation to price a European call option with a strike price ($K = 105$) and a risk-free rate ($r = 0.01$).

Aim:

To estimate the theoretical fair value of a European call option by calculating the discounted expected payoff across multiple simulated terminal stock price scenarios.

Procedure:

1. Define an option pricing function that computes the payoff for multiple simulation paths.
2. Average these payoffs.
3. Discount the result to the present value using the risk-free rate.

Program:

```
S0 <- 100
mu <- 0.001
sigma <- 0.02
T <- 252
n_sims <- 1000
option_price <- function(S0, K, mu, sigma, T, n_sims, r){
  terminal_prices <- simulate_terminal(S0, mu, sigma, T, n_sims)
  payoffs <- pmax(terminal_prices - K, 0)
  mean(payoffs) * exp(-r * T / 252)
}
option_price(S0, 105, mu, sigma, T, n_sims, 0.01)
```

Output:

33.22924

10.Problem Statement:

Simulate portfolio returns for 3 assets and find the optimal allocation.

Aim:

To evaluate portfolio performance by simulating asset returns and calculating the expected return and risk associated with a specific asset allocation strategy.

Procedure:

1. Define a portfolio simulation function incorporating asset weights and market parameters.
2. Execute the simulation across multiple trials to generate return data.
3. Calculate the mean and standard deviation to assess the expected return and portfolio risk.

Program:

```
S0 <- 100
mu <- 0.001
sigma <- 0.02
T <- 252
n_sims <- 1000
simulate_portfolio <- function(weights, mus, sigmas, T, n_sims) {
  portfolio_returns <- replicate(n_sims, {
    prices <- sapply(1:length(mus), function(i) simulate_stock(100, mus[i], sigmas[i], T))
    portfolio_value <- rowSums(t(t(prices) * weights))
    (portfolio_value[T] - portfolio_value) / portfolio_value
  })
  return(portfolio_returns)
}

weights <- c(0.4, 0.4, 0.2)
mus <- c(0.001, 0.002, 0.0008)
sigmas <- c(0.02, 0.03, 0.015)
portfolio_returns <- simulate_portfolio(weights, mus, sigmas, T, n_sims)
mean(portfolio_returns)
sd(portfolio_returns)
```

Output:

```
0.2434599
0.2949104
```

Signature of the faculty

Ex. No.10	X. READING, WRITING DATA IN R AND WORKING WITH INBUILT DATA SETS
Date : 11.04.2026	

1.Problem Statement:

Read a CSV file containing data on student scores and display the first 5 rows.

Aim:

To learn how to import data from a CSV file into R and view a subset of the data using built-in functions.

Procedure:

1. Create or obtain a CSV file named student_scores.csv containing student data (Name, Subject, Score).
2. Open R or RStudio.
3. Use the read.csv() function to load the CSV file into a variable.
4. Use the head() function to display the first five rows of the dataset.
5. Verify that the data has been loaded correctly.

Program:

```
data <- read.csv("student_scores.csv")  
head(data, 5)
```

Output:

Name	Subject	Score
John	Math	85
Alice	Science	92
Bob	Math	78
Clara	English	88
Eve	Science	90

2.Problem Statement:

To create a data frame containing employee details and save it as a CSV file.

Aim:

To learn how to create a data frame in R and export it to a CSV file.

Procedure:

1. Open R or RStudio.
2. Create a data frame using the data.frame() function with employee details.
3. Use the write.csv() function to save the data frame as a CSV file.
4. Specify row.names = FALSE to avoid writing row numbers.
5. Verify that the file is created in the working directory.

Program:

```
employee_data <- data.frame(  
  ID = c(1, 2, 3, 4),  
  Name = c("John", "Alice", "Bob", "Clara"),  
  Salary = c(50000, 60000, 55000, 58000)  
)  
  
write.csv(employee_data, "employee_data.csv", row.names = FALSE)  
  
print("File written successfully")
```

Output:

ID	Name	Salary
1	John	50000
2	Alice	60000
3	Bob	55000
4	Clara	58000

3.Problem Statement:

To load the inbuilt dataset *iris* and display its structure.

Aim:

To learn how to load and explore built-in datasets in R using appropriate functions.

Procedure:

1. Open R or RStudio.
2. Use the `data()` function to load the *iris* dataset.
3. Use the `str()` function to view the structure of the dataset.
4. Observe the number of observations, variables, and data types.

Program:

```
data(iris)  
  
str(iris)
```

Output:

```
'data.frame': 150 obs. of 5 variables:  
 $ Sepal.Length: num 5.1 4.9 4.7 4.6 5 ...  
 $ Sepal.Width : num 3.5 3.0 3.2 3.1 3.6 ...  
 $ Petal.Length: num 1.4 1.4 1.3 1.5 1.4 ...  
 $ Petal.Width : num 0.2 0.2 0.2 0.2 0.2 ...  
 $ Species      : Factor w/ 3 levels "setosa","versicolor","virginica": 1 1 1 1 1 ...
```

4.Problem Statement:

To generate summary statistics for the inbuilt dataset *mtcars*.

Aim:

To learn how to obtain statistical summaries of a dataset using built-in functions in R.

Procedure:

1. Open R or RStudio.
2. Load the *mtcars* dataset using the `data()` function.
3. Use the `summary()` function to generate summary statistics.
4. Observe measures like minimum, maximum, median, and quartiles

Program:

```
data(mtcars)

summary(mtcars)
```

Output:

mpg	cyl	disp	hp
Min. :10.40	Min. :4.000	Min. : 71.1	Min. : 52.0
1st Qu.:15.43	1st Qu.:4.000	1st Qu.:120.8	1st Qu.: 96.5
Median :19.20	Median :6.000	Median :196.3	Median :123.0
Mean :20.09	Mean :6.188	Mean :230.7	Mean :146.7
3rd Qu.:22.80	3rd Qu.:8.000	3rd Qu.:326.0	3rd Qu.:180.0
Max. :33.90	Max. :8.000	Max. :472.0	Max. :335.0

5.Problem Statement:

To write a data frame to an Excel file.

Aim:

To learn how to export a data frame from R to an Excel file using external packages.

Procedure:

1. · Open R or RStudio.
2. · Install the `writexl` package if it is not already installed.
3. · Load the package using the `library()` function.
4. · Use the `write_xlsx()` function to export the data frame to an Excel file.
5. · Verify that the Excel file is created in the working directory.

Program:

```
if (!require("writexl")) install.packages("writexl")

library(writexl)

write_xlsx(employee_data, "employee_data.xlsx")

print("Excel file written successfully")
```

Output:

A file named **employee_data.xlsx** is created in the working directory.

6.Problem Statement:

To filter rows in the *mtcars* dataset where mileage (mpg) is greater than 20.

Aim:

To learn how to extract specific rows from a dataset based on given conditions.

Procedure:

1. Open R or RStudio.
2. Load the *mtcars* dataset using the `data()` function (if not already loaded).
3. Use the `subset()` function to filter rows where `mpg > 20`.
4. Store the result in a new variable.
5. Display the filtered dataset.

Program:

```
data(mtcars)

filtered_data <- subset(mtcars, mpg > 20)

print(filtered_data)
```

Output:

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1

7.Problem Statement:

To read data from a URL containing a CSV file and display the first five rows.

Aim:

To learn how to import data directly from an online source into R.

Procedure:

1. Open R or RStudio.
2. Specify the URL containing the CSV dataset.
3. Use the `read.csv()` function to load the data from the URL.
4. Store the dataset in a variable.
5. Use the `head()` function to display the first five rows.

Program:

```
url <- "https://people.sc.fsu.edu/~jburkardt/data/csv/hw_200.csv"

data <- read.csv(url)

head(data, 5)
```

Output:

```
Height.Inches. Weight.Pounds.
1      65.783      112.992
2      71.515      136.487
3      69.398      153.027
4      68.216      142.335
5      67.787      144.297
```

8.Problem Statement:

To add new rows to an existing data frame.

Aim:

To learn how to append new observations to a data frame using R functions.

Procedure:

1. Open R or RStudio.
2. Create a new data frame containing additional rows.
3. Use the `rbind()` function to append the new rows to the existing data frame.
4. Store the updated data in a new variable.
5. Display the updated data frame.

Program:

```
new_rows <- data.frame(
  ID = c(5, 6),
  Name = c("Eve", "Mark"),
  Salary = c(61000, 53000)
)

updated_data <- rbind(employee_data, new_rows)

print(updated_data)
```

Output:

```
  ID Name Salary
1  1 John  50000
2  2 Alice 60000
3  3 Bob   55000
4  4 Clara 58000
5  5 Eve   61000
6  6 Mark  53000
```

9.Problem Statement:

To save a data frame as an *.rds* file and load it back into R.

Aim:

To learn how to store and retrieve R objects using the RDS file format.

Procedure:

1. Open R or RStudio.
2. Use the `saveRDS()` function to save the data frame as an *.rds* file.
3. Use the `readRDS()` function to load the saved file back into R.
4. Store the loaded data in a new variable.
5. Display the loaded data to verify correctness.

Program:

```
saveRDS(employee_data, "employee_data.rds")

loaded_data <- readRDS("employee_data.rds")

print(loaded_data)
```

Output:

The data frame is successfully saved as **employee_data.rds** and loaded back into R. The loaded data is displayed correctly.

10.Problem Statement:

To merge two data frames based on a common column.

Aim:

To learn how to combine two data frames using a common key in R.

Procedure:

1. Open R or RStudio.
2. Create two data frames with a common column.
3. Use the `merge()` function to combine the data frames.
4. Specify the common column using the `by` argument.
5. Display the merged data.

Program:

```
df1 <- data.frame(ID = c(1, 2, 3), Name = c("John", "Alice", "Bob"))

df2 <- data.frame(ID = c(1, 2, 3), Department = c("HR", "IT", "Finance"))

merged_data <- merge(df1, df2, by = "ID")

print(merged_data)
```

Output:

	ID	Name	Department
1	1	John	HR
2	2	Alice	IT
3	3	Bob	Finance

Signature of the faculty